Chapter 12. Native Methods

In this chapter

- 12.1 Calling a C Function from a Java Program
- 12.2 Numeric Parameters and Return Values
- 12.3 String Parameters
- 12.4 Accessing Fields
- 12.5 Encoding Signatures
- 12.6 Calling Java Methods
- 12.7 Accessing Array Elements
- 12.8 Handling Errors
- 12.9 Using the Invocation API
- 12.10 A Complete Example: Accessing the Windows Registry
- 12.11 Foreign Functions: A Glimpse into the Future

While a "100% Pure Java" solution is nice in principle, there are situations in which you will want to write (or use) code in another language. Such code is usually called *native* code.

Particularly in the early days of Java, many people assumed that it would be a good idea to use C or C++ to speed up critical parts of a Java application. However, in practice, this was rarely useful. A presentation at the 1996 JavaOne conference showed this clearly. The developers of the cryptography library at Sun Microsystems reported that a pure Java platform implementation of their cryptographic functions was more than adequate. It was true that the code was not as fast as a C implementation would have

been, but it turned out not to matter. The Java platform implementation was far faster than the network I/O. This turned out to be the real bottleneck.

Of course, there are drawbacks to going native. If a part of your application is written in another language, you must supply a separate native library for every platform you want to support. Code written in C or C++ offers no protection against overwriting memory through invalid pointer usage. It is easy to write native methods that corrupt your program or infect the operating system.

Thus, I suggest using native code only when you need to. In particular, there are three reasons why native code might be the right choice:

- Your application requires access to system features or devices that are not accessible through the Java platform.
- You have substantial amounts of tested and debugged code in another language, and you know how to port it to all desired target platforms.
- You have found, through benchmarking, that the Java code is much slower than the equivalent code in another language.

The Java platform has an API for interoperating with native C code called the Java Native Interface (JNI). We'll discuss JNI programming in this chapter.



C++ Note

You can also use C++ instead of C to write native methods. There are a few advantages—type checking is slightly stricter, and accessing the JNI functions is a bit more convenient. However, JNI does not support any mapping between Java and C++ classes.

As you will see, there is a certain amount of tedium involved in providing a binding layer between Java and native code. Java 17 has, as a preview feature, an API to access "foreign" functions and memory that is quite a bit more convenient than JNI. At the end

of this chapter, you will get a glimpse into this API.

12.1 Calling a C Function from a Java Program

Suppose you have a C function that does something you like and, for one reason or another, you don't want to bother reimplementing it in Java. For the sake of illustration, we'll start with a simple C function that prints a greeting.

The Java programming language uses the keyword

```
native
```

for a native method, and you will obviously need to place a method in a class. The result is shown in **Listing 12.1**.

The

```
native
```

keyword alerts the compiler that the method will be defined externally. Of course, native methods will contain no Java code, and the method header is followed immediately by a terminating semicolon. Therefore, native method declarations look similar to abstract method declarations.

Listing 12.1

helloNative/HelloNative.java

Click here to view code image

```
1 /**
2  * @version 1.11 2007-10-26
3  * @author Cay Horstmann
4  */
5 class HelloNative
6 {
```

```
7    public static native void greeting();
8 }
```



Note

We do not use packages here to keep examples simple.

In this particular example, the native method is also declared as `static`. Native methods can be both static and nonstatic. We'll start with a static method because we do not yet want to deal with parameter passing.

You can actually compile this class, but if you try to use it in a program, the virtual machine will tell you it doesn't know how to find `greeting`—reporting an `UnsatisfiedLinkError`. To implement the native code, write a corresponding C function. You must name that function *exactly* the way the Java virtual machine expects. Here are the rules:

- Use the full Java method name, such as `HelloNative.greeting`. If the class is in a package, prepend the package name, such as `com.horstmann.HelloNative.greeting`.

Replace every period with an underscore, and append the prefix

```
Java_
```

. For example,

```
Java_HelloNative_greeting
```

or

```
Java_com_horstmann_HelloNative_greeting
```

.

If the class name contains characters that are not ASCII letters or digits—that is,

```
'_'
```

,

```
'$'
```

, or Unicode characters with codes greater than

```
\u007F
```

—replace them with

```
_0xxxx
```

, where

```
xxxx
```

is the sequence of four hexadecimal digits of the character's Unicode value.



Note

If you *overload* native methods—that is, if you provide multiple native methods with the same name—you must append a double underscore followed by the encoded argument types. (I'll describe the encoding of the argument types later in this chapter.) For example, if you have a native method

```
greeting
```

and another native method

`greeting(int repeat)`

, then the first one is called

`Java_HelloNative_greeting__`

and the second,

`Java_HelloNative_greeting__I`

.

Actually, nobody does this by hand; instead, run

`javac`

with the

`-h`

flag, providing the directory in which the header files should be placed:

```
javac -h . HelloNative.java
```

This command creates a header file

`HelloNative.h`

in the current directory, as shown in **Listing 12.2**.

Listing 12.2

helloNative/HelloNative.h

Click here to view code image

```
/* DO NOT EDIT THIS FILE - it is machine generated */
#include <jni.h>
/* Header for class HelloNative */

#ifndef _Included_HelloNative
#define _Included_HelloNative
#ifdef __cplusplus
extern "C" {
#endif
/*
 * Class: HelloNative
 * Method: greeting
 * Signature: ()V
 */
JNIEXPORT void JNICALL Java_HelloNative_greeting
  (JNIEnv *, jclass);

#ifdef __cplusplus
}
#endif
#endif
```

As you can see, this file contains the declaration of a function

`Java_HelloNative_greeting`

. (The macros

`JNIEXPORT`

and

JNICALL

are defined in the header file

jni.h

. They denote compiler-dependent specifiers for exported functions that come from a dynamically loaded library.)

Now, simply copy the function prototype from the header file into a source file and give the implementation code for the function, as shown in **Listing 12.3**.

Listing 12.3

helloNative/HelloNative.c

Click here to view code image

```
 1 /*
 2   @version 1.10 1997-07-01
 3   @author Cay Horstmann
 4 */
 5
 6 #include "HelloNative.h"
 7 #include <stdio.h>
 8
 9 JNIEXPORT void JNICALL Java_HelloNative_greeting(JNIEnv* env, jclass cl)
10 {
11    printf("Hello, Native World!\n");
12 }
```

In this simple function, ignore the

`env`

and

`cl`

arguments. You'll see their use later.



C++ Note

You can use C++ to implement native methods. However, you must then declare the functions that implement the native methods as `extern "C"`. (This stops the C++ compiler from "mangling" the method name.) For example,

Click here to view code image

```
extern "C"

JNIEXPORT void JNICALL Java_HelloNative_greeting(JNIEnv* env, jclass cl)
{
  cout << "Hello, Native World!" << endl;
}
```

Compile the native C code into a dynamically loaded library. The details depend on your compiler.

For example, with the GNU C compiler on Linux, use these commands:

Click here to view code image

```
gcc -fPIC -I
jdk
/include -I
jdk
/include/linux -shared -o libHelloNative.so HelloNative.c
```

With the Microsoft compiler under Windows, the command is

Click here to view code image

```
cl -I
jdk
\include -I
jdk
\include\win32 -LD HelloNative.c -FeHelloNative.dll
```

Here,

jdk

is the directory that contains the JDK.



Tip

If you use the Microsoft compiler from a command shell, first run a batch file such as

```
vsvars32.bat
```

or

```
vcvarsall.bat
```

. That batch file sets up the path and the environment variables needed by the compiler. You can find it in the directory

```
c:\Program Files\Microsoft Visual Studio 14.0\Common7\Tools
```

or a similar monstrosity. Check the Visual Studio documentation for details.

You can also use the freely available Cygwin programming environment from

```
www.cygwin.com
```

. It contains the GNU C compiler and libraries for UNIX-style programming on Windows. With Cygwin, use the command

Click here to view code image

```
gcc -mno-cygwin -D _int64="long long" -I
jdk
/include/ -I
jdk
/include/win32 \
   -shared -Wl,--add-stdcall-alias -o HelloNative.dll HelloNative.c
```



Note

The Windows version of the header file

`jni_md.h`

contains the type declaration

```
typedef __int64 jlong;
```

which is specific to the Microsoft compiler. If you use the GNU compiler, you might want to edit that file, for example,

Click here to view code image

```
#ifdef __GNUC__
  typedef long long jlong;
#else
  typedef __int64 jlong;
#endif
```

Alternatively, compile with

`-D __int64="long long"`

, as shown in the sample compiler invocation.

Finally, add a call to the

`System.loadLibrary`

method in your program. To ensure that the virtual machine will load the library before the first use of the class, use a static initialization block, as in **Listing 12.4**.

Figure 12.1 gives a summary of the native code processing.

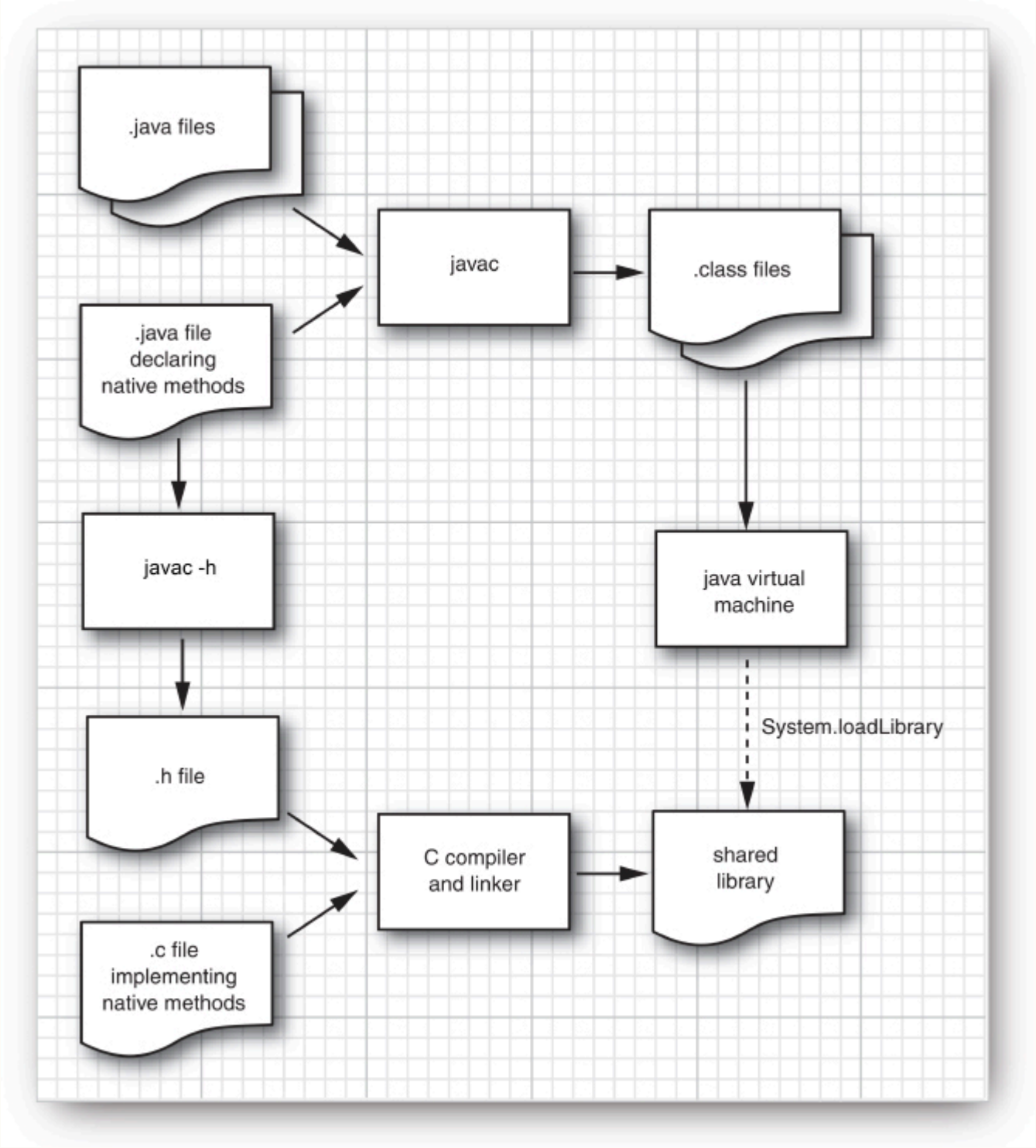

.java files
javac
.class files
.java file
declaring
native methods
javac -h
java virtual
machine
System.loadLibrary
.h file
C compiler
and linker
shared
library
.c file
implementing
native methods

Figure 12.1 Processing native code

Listing 12.4

helloNative/HelloNativeTest.java

Click here to view code image

```
 1 /**
 2  * @version 1.11 2007-10-26
 3  * @author Cay Horstmann
 4  */
 5 class HelloNativeTest
 6 {
 7    public static void main(String[] args)
 8    {
 9       HelloNative.greeting();
10    }
11
12    static
13    {
14       System.loadLibrary("HelloNative");
```

```
15     }
16 }
```

After you compile and run this program, the message “Hello, Native World!” is displayed in a terminal window.

 **Note**

If you run Linux, you must add the current directory to the library path. Either set the `LD_LIBRARY_PATH` environment variable:

Click here to view code image

```
export LD_LIBRARY_PATH=.:$LD_LIBRARY_PATH
```

or set the `java.library.path` system property:

Click here to view code image

```
java -Djava.library.path=. HelloNativeTest
```

Of course, this is not particularly impressive by itself. Keep in mind, however, that this message is generated by the C `printf` command and not by any Java code. We have taken the first step toward bridging the gap between the two languages!

In summary, follow these steps to link a native method to a Java program:

1. Declare a native method in a Java class.
2. Run `javac -h` to get a header file with a C declaration for the method.
3. Implement the native method in C.
4. Place the code in a shared library.
5. Load that library in your Java program.

`java.lang.System`

1.0

`void loadLibrary(String libname)`

loads the library with the given name. The library is located in the library search path. The exact method for locating the library depends on the operating system.



Note

Some shared libraries for native code must execute certain initializations. You can place any initialization code into a

`JNI_OnLoad`

method. Similarly, when the virtual machine (VM) shuts down, it will call the

`JNI_OnUnload`

method if you provide it. The prototypes are

Click here to view code image

```
jint JNI_OnLoad(JavaVM* vm, void* reserved);
void JNI_OnUnload(JavaVM* vm, void* reserved);
```

The

`JNI_OnLoad`

method needs to return the minimum version of the VM it requires, such as

`JNI_VERSION_1_2`

.

12.2 Numeric Parameters and Return Values

When passing numbers between C and Java, you should understand which types correspond to each other. For example, although C does have data types called

`int`

and

`long`

, their implementation is platform-dependent. On some platforms, an

`int`

is a 16-bit quantity, on others it is a 32-bit quantity. On the Java platform, of course, an

`int`

is *always* a 32-bit integer. For that reason, JNI defines types

`jint`

,

`jlong`

, and so on.

Table 12.1 shows the correspondence between Java types and C types.

Table 12.1 Java Types and C Types

Java Programming Language	C Programming Language	Bytes
boolean	jboolean	1
byte	jbyte	1

Java Programming Language	C Programming Language	Bytes
char	jchar	2
short	jshort	2
int	jint	4
long	jlong	8
float	jfloat	4
double	jdouble	8

In the header file

`jni.h`

, these types are declared with

`typedef`

statements as the equivalent types on the target platform. That header file also defines the constants

`JNI_FALSE = 0`

and

`JNI_TRUE = 1`

.

Until Java 5, Java had no direct analog of the C

printf

function. In the following examples, we will pretend you are stuck with an ancient JDK release and decide to implement the same functionality by calling the C

printf

function in a native method.

Listing 12.5 shows a class called

Printf1

that uses a native method to print a floating-point number with a given field width and precision.

Listing 12.5

printf1/Printf1.java

Click here to view code image

```
 1 /**
 2  * @version 1.10 1997-07-01
 3  * @author Cay Horstmann
 4  */
 5 class Printf1
 6 {
 7    public static native int print(int width, int precision, double x);
 8
 9    static
10    {
11       System.loadLibrary("Printf1");
```

```
12    }
13 }
```

Notice that when the method is implemented in C, all

`int`

and

`double`

parameters are changed to

`jint`

and

`jdouble`

, as shown in **Listing 12.6**.

Listing 12.6

printf1/Printf1.c

Click here to view code image

```
1 /**
2    @version 1.10 1997-07-01
3    @author Cay Horstmann
4 */
5
6 #include "Printf1.h"
7 #include <stdio.h>
8
```

```
 9 JNIEXPORT jint JNICALL Java_Printf1_print(JNIEnv* env, jclass cl,
10       jint width, jint precision, jdouble x)
11 {
12    char fmt[30];
13    jint ret;
14    sprintf(fmt, "%%%d.%df", width, precision);
15    ret = printf(fmt, x);
16    fflush(stdout);
17    return ret;
18 }
```

The function simply assembles a format string

"%w.*p*f"

in the variable

fmt

, then calls

printf

. It returns the number of characters printed.

Listing 12.7 shows the test program that demonstrates the

Printf1

class.

Listing 12.7

printf1/Printf1Test.java

Click here to view code image

```
 1 /**
 2  * @version 1.10 1997-07-01
 3  * @author Cay Horstmann
 4  */
 5 class Printf1Test
 6 {
 7    public static void main(String[] args)
 8    {
 9       int count = Printf1.print(8, 4, 3.14);
10       count += Printf1.print(8, 4, count);
11       System.out.println();
12       for (int i = 0; i < count; i++)
13          System.out.print("-");
14       System.out.println();
15    }
16 }
```

12.3 String Parameters

Next, let's look at how to transfer strings to and from native methods. Strings are quite different in the two languages: In Java, they are sequences of UTF-16 code points, whereas C strings are null-terminated sequences of bytes. JNI has two sets of functions for manipulating strings: One converts Java strings to "modified UTF-8" byte sequences and another converts them to arrays of UTF-16 values—that is, to

```
jchar
```

arrays. (The UTF-8, "modified UTF-8," and UTF-16 formats were discussed in **Chapter 2**. Recall that the UTF-8 and "modified UTF-8" encodings leave ASCII characters unchanged, but all other Unicode characters are encoded as multibyte sequences.)



Note

The standard UTF-8 encoding and the “modified UTF-8” encoding differ only for “supplementary” characters with codes higher than

```
OxFFFF
```

. In the standard UTF-8 encoding, these characters are encoded as 4-byte sequences. In the “modified” encoding, each such character is first encoded as a pair of “surrogates” in the UTF-16 encoding, and then each surrogate is encoded with UTF-8, yielding a total of 6 bytes. This is clumsy, but it is a historical accident—the JVM specification was written when Unicode was still limited to 16 bits.

If your C code already uses Unicode, you'll want to use the second set of conversion functions. On the other hand, if all your strings are restricted to ASCII characters, you can use the “modified UTF-8” conversion functions.

A native method with a

```
String
```

parameter actually receives a value of an opaque type called

```
jstring
```

. A native method with a return value of type

```
String
```

must return a value of type

```
jstring
```

. JNI functions read and construct these

```
jstring
```

objects. For example, the

```
NewStringUTF
```

function makes a new

`jstring`

object out of a

`char`

array that contains ASCII characters or, more generally, “modified UTF-8”-encoded byte sequences.

JNI functions have a somewhat odd calling convention. Here is a call to the

`NewStringUTF`

function:

Click here to view code image

```
JNIEXPORT jstring JNICALL Java_HelloNative_getGreeting(JNIEnv* env, jclass cl)
{
  jstring jstr;
  char greeting[] = "Hello, Native World\n";
  jstr = (*
env
)->NewStringUTF(
env
, greeting);
  return jstr;
}
```



Note

Unless explicitly mentioned otherwise, all code in this chapter is C code.

All calls to JNI functions use the

`env`

pointer that is the first argument of every native method. The

`env`

pointer is a pointer to a table of function pointers (see **Figure 12.2**). Therefore, you must prefix every JNI call with

`(*env)->`

to actually dereference the function pointer. Furthermore,

`env`

is the first parameter of every JNI function.

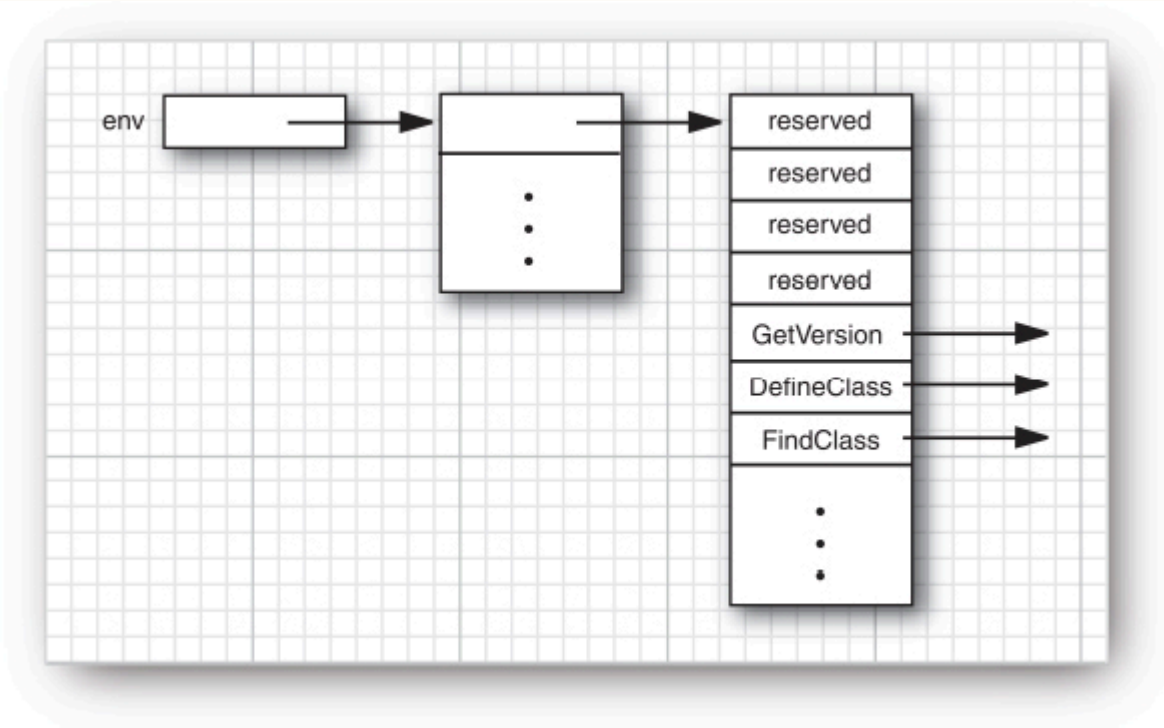


Figure 12.2 The `env` pointer



C++ Note

It is simpler to access JNI functions in C++. The C++ version of the `JNIEnv` class has inline member functions that take care of the function pointer lookup for you. For example, you can call the `NewStringUTF` function as

Click here to view code image

```
jstr = env->NewStringUTF(greeting);
```

Note that you omit the `JNIEnv`

pointer from the parameter list of the call.

The
`NewStringUTF`
function lets you construct a new
`jstring`
. To read the contents of an existing
`jstring`
object, use the
`GetStringUTFChars`
function. This function returns a
`const jbyte*`
pointer to the “modified UTF-8” characters that describe the character string. Note that a specific virtual machine is free to choose this character encoding for its internal string representation, so you might get a character pointer into the actual Java string. Since Java strings are meant to be immutable, it is *very* important that you treat the
`const`
seriously and do not try to write into this character array. On the other hand, if the virtual machine uses UTF-16 or UTF-32 characters for its internal string representation, this function call allocates a new memory block that will be filled with the “modified UTF-8” equivalents.

The virtual machine must know when you are finished using the string so that it can garbage-collect it. (The garbage collector runs in a separate thread, and it can interrupt the execution of native methods.) For that reason, you must call the
`ReleaseStringUTFChars`
function.

Alternatively, you can supply your own buffer to hold the string characters by calling the

GetStringRegion

or

GetStringUTFRegion

methods.

Finally, the

GetStringUTFLength

function returns the number of characters needed for the “modified UTF-8” encoding of the string.

 **Note**

You can find the JNI API at

http://docs.oracle.com/javase/7/docs/technotes/guides/jni

.

Accessing Java Strings from C Code

jstring NewStringUTF(JNIEnv* env, const char bytes[])

returns a new Java string object from a zero byte-terminated “modified UTF-8” byte sequence, or

NULL

if the string cannot be constructed.

jsize GetStringUTFLength(JNIEnv* env, jstring string)

returns the number of bytes required for the “modified UTF-8” encoding (not counting the zero byte terminator).

const jbyte* GetStringUTFChars(JNIEnv* env, jstring string, jboolean* isCopy)

returns a pointer to the “modified UTF-8” encoding of a string, or

```
NULL
```

if the character array cannot be constructed. The pointer is valid until

```
ReleaseStringUTFChars
```

is called.

```
isCopy
```

points to a

```
jboolean
```

filled with

```
JNI_TRUE
```

if a copy is made, or with

```
JNI_FALSE
```

otherwise.

```
void ReleaseStringUTFChars(JNIEnv* env, jstring string, const jbyte bytes[])
```

informs the virtual machine that the native code no longer needs access to the Java string through

```
bytes
```

(a pointer returned by

```
GetStringUTFChars
```

).

```
void GetStringRegion(JNIEnv *env, jstring string, jsize start, jsize length, jchar *buffer)
```

copies a sequence of UTF-16 double bytes from a string to a user-supplied buffer of size at least 2 ×

```
length
```

.

```
void GetStringUTFRegion(JNIEnv *env, jstring string, jsize start, jsize length, jbyte *buffer)
```

copies a sequence of “modified UTF-8” bytes from a string to a user-supplied buffer. The buffer must be long enough to hold the bytes. In the worst case, 3 ×

`length`

bytes are copied.

`jstring NewString(JNIEnv* env, const jchar chars[], jsize length)`

returns a new Java string object from a Unicode string, or

`NULL`

if the string cannot be constructed.

`jsize GetStringLength(JNIEnv* env, jstring string)`

returns the number of characters in the string.

`const jchar* GetStringChars(JNIEnv* env, jstring string, jboolean* isCopy)`

returns a pointer to the Unicode encoding of a string, or

`NULL`

if the character array cannot be constructed. The pointer is valid until

`ReleaseStringChars`

is called.

`isCopy`

is either

`NULL`

or points to a

`jboolean`

filled with

`JNI_TRUE`

if a copy is made, or with

`JNI_FALSE`

otherwise.

`void ReleaseStringChars(JNIEnv* env, jstring string, const jchar chars[])`

informs the virtual machine that the native code no longer needs access to the Java string through

`chars`

(a pointer returned by

`GetStringChars`

).

Let us put these functions to work and write a class that calls the C function

`sprintf`

. We would like to call the function as shown in **Listing 12.8**.

Listing 12.8

printf2/Printf2Test.java

Click here to view code image

```
1 /**
2  * @version 1.10 1997-07-01
3  * @author Cay Horstmann
4  */
5 class Printf2Test
6 {
7    public static void main(String[] args)
8    {
9       double price = 44.95;
```

```
10       double tax = 7.75;
11       double amountDue = price * (1 + tax / 100);
12
13       String s = Printf2.sprint("Amount due = %8.2f", amountDue);
14       System.out.println(s);
15    }
16 }
```

Listing 12.9 shows the class with the native

sprint

method.

Listing 12.9

printf2/Printf2.java

Click here to view code image

```
 1 /**
 2  * @version 1.10 1997-07-01
 3  * @author Cay Horstmann
 4  */
 5 class Printf2
 6 {
 7    public static native String sprint(String format, double x);
 8
 9    static
10    {
11       System.loadLibrary("Printf2");
```

```
12     }
13 }
```

Therefore, the C function that formats a floating-point number has the prototype

Click here to view code image

```
JNIEXPORT jstring JNICALL Java_Printf2_sprint(JNIEnv* env, jclass cl,
    jstring format, jdouble x)
```

Listing 12.10 shows the code for the C implementation. Note the calls to

`GetStringUTFChars`

to read the format argument,

`NewStringUTF`

to generate the return value, and

`ReleaseStringUTFChars`

to inform the virtual machine that access to the string is no longer required.

Listing 12.10

printf2/Printf2.c

Click here to view code image

```
/**
   @version 1.10 1997-07-01
   @author Cay Horstmann
*/

#include "Printf2.h"
#include <string.h>
#include <stdlib.h>
#include <float.h>

/**
   @param format a string containing a printf format specifier
   (such as "%8.2f"). Substrings "%%" are skipped.
   @return a pointer to the format specifier (skipping the '%')
   or NULL if there wasn't a unique format specifier
*/
char* find_format(const char format[])
{
   char* p;
   char* q;

   p = strchr(format, '%');
   while (p != NULL && *(p + 1) == '%') /* skip %% */
      p = strchr(p + 2, '%');
   if (p == NULL) return NULL;
   /* now check that % is unique */
   p++;
   q = strchr(p, '%');
   while (q != NULL && *(q + 1) == '%') /* skip %% */
      q = strchr(q + 2, '%');
   if (q != NULL) return NULL; /* % not unique */
```

```
   q = p + strspn(p, " -0+#"); /* skip past flags */
   q += strspn(q, "0123456789"); /* skip past field width */
   if (*q == '.') { q++; q += strspn(q, "0123456789"); }
      /* skip past precision */
   if (strchr("eEfFgG", *q) == NULL) return NULL;
      /* not a floating-point format */
   return p;
}

JNIEXPORT jstring JNICALL Java_Printf2_sprint(JNIEnv* env, jclass cl,
      jstring format, jdouble x)
{
   const char* cformat;
   char* fmt;
   jstring ret;

   cformat = (*env)->GetStringUTFChars(env, format, NULL);
   fmt = find_format(cformat);
   if (fmt == NULL)
      ret = format;
   else
   {
      char* cret;
      int width = atoi(fmt);
      if (width == 0) width = DBL_DIG + 10;
      cret = (char*) malloc(strlen(cformat) + width);
      sprintf(cret, cformat, x);
      ret = (*env)->NewStringUTF(env, cret);
      free(cret);
    }
    (*env)->ReleaseStringUTFChars(env, format, cformat);
```

```
63     return ret;
64 }
```

In this function, we chose to keep error handling simple. If the format code to print a floating-point number is not of the form `%w.pc`, where `c` is one of the characters `e`, `E`, `f`, `g`, or `G`, then we simply do not format the number. We'll show you later how to make a native method throw an exception.

12.4 Accessing Fields

All the native methods you saw so far were static methods with number and string parameters. We'll now consider native methods that operate on objects. As an exercise, we will reimplement as native a method of the `Employee`

class that was introduced in Volume I, Chapter 4. Again, this is not something you would normally want to do, but it does illustrate how to access fields from a native method when you need to do so.

12.4.1 Accessing Instance Fields

To see how to access instance fields from a native method, we will reimplement the

`raiseSalary`

method. Here is the code in Java:

Click here to view code image

```
public void raiseSalary(double byPercent)
{
  salary *= 1 + byPercent / 100;
}
```

Let us rewrite this as a native method. Unlike the previous examples of native methods, this is not a static method. Running

`javac -h`

gives the following prototype:

Click here to view code image

```
JNIEXPORT void JNICALL Java_Employee_raiseSalary(JNIEnv *, jobject, jdouble);
```

Note the second argument. It is no longer of type

`jclass`

but of type

`jobject`

. In fact, it is an equivalent of the

`this`

reference. Static methods obtain a reference to the class, whereas nonstatic methods obtain a reference to the implicit

`this`

argument object.

Now we access the

`salary`

field of the implicit argument. In the "raw" Java-to-C binding of Java 1.0, this was easy—a programmer could directly access instance fields. However, direct access requires all virtual machines to expose their internal data layout. For that reason, the JNI requires programmers to get and set the values of instance fields by calling special JNI functions.

In our case, we need to use the

`GetDoubleField`

and

`SetDoubleField`

functions because the type of

`salary`

is

`double`

. There are other functions—

`GetIntField/SetIntField`

,

`GetObjectField/SetObjectField`

, and so on for other field types. The general syntax is:

Click here to view code image

```
x = (*env)->Get
Xxx
Field(env, this_obj, fieldID);
(*env)->Set
Xxx
Field(env, this_obj, fieldID, x);
```

Here,

`fieldID`

is a value of a special type,

`jfieldID`

, that identifies a field in a structure, and

`Xxx`

represents a Java data type (

`Object`

,

`Boolean`

,

`Byte`

, and so on). To obtain the

`fieldID`

, you must first get a value representing the class, which you can do in one of two ways. The

`GetObjectClass`

function returns the class of any object. For example:

Click here to view code image

```
jclass class_Employee = (*env)->GetObjectClass(env, this_obj);
```

The

`FindClass`

function lets you specify the class name as a string (curiously, with

`/`

characters instead of periods as package name separators).

Click here to view code image

```
jclass class_String = (*env)->FindClass(env,
"java/lang/String"
);
```

Use the

GetFieldID

function to obtain the

fieldID

. You must supply the name of the field and its *signature*, an encoding of its type. For example, here is the code to obtain the field ID of the

salary

field:

Click here to view code image

```
jfieldID id_salary = (*env)->GetFieldID(env, class_Employee, "salary", "D");
```

The string

"D"

denotes the type

double

. You'll learn the complete rules for encoding signatures in the next section.

You might be thinking that accessing an instance field is quite convoluted. The designers of the JNI did not want to expose the instance fields directly, so they had to supply functions for getting and setting field values. To minimize the cost of these functions, computing the field ID from the field name—which is the most expensive step—is factored out into a separate step. That is, if you repeatedly get and set the value of a particular field, you can incur the cost of computing the field identifier only once.

Let us put all the pieces together. The following code reimplements the

raiseSalary

method as a native method:

Click here to view code image

```
JNIEXPORT void JNICALL Java_Employee_raiseSalary(JNIEnv* env, jobject this_obj,
    jdouble byPercent)
{
  /* get the class */
  jclass class_Employee = (*env)->GetObjectClass(env, this_obj);

  /* get the field ID */
  jfieldID id_salary = (*env)->GetFieldID(env, class_Employee, "salary", "D");

  /* get the field value */
  jdouble salary = (*env)->GetDoubleField(env, this_obj, id_salary);

  salary *= 1 + byPercent / 100;

  /* set the field value */
  (*env)->SetDoubleField(env, this_obj, id_salary, salary);
}
```



Caution

Class references are only valid until the native method returns. You cannot cache the return values of `GetObjectClass` in your code. Do *not* store away a class reference for reuse in a later method call. You must call

`GetObjectClass`

every time the native method executes. If this is intolerable, you can lock the reference with a call to

`NewGlobalRef`

:

Click here to view code image

```
static jclass class_X = 0;
static jfieldID id_a;
. . .
if (class_X == 0)
{
   jclass cx = (*env)->GetObjectClass(env, obj);
   class_X = (*env)->NewGlobalRef(env, cx);
   id_a = (*env)->GetFieldID(env, class_X, "a", ". . .");
}
```

Now you can use the class reference and field IDs in subsequent calls. When you are done using the class, make sure to call

```
(*env)->DeleteGlobalRef(env, class_X);
```

Listings 12.11 and 12.12 show the Java code for a test program and the

Employee

class. **Listing 12.13** contains the C code for the native

raiseSalary

method.

Listing 12.11

employee/EmployeeTest.java

Click here to view code image

```
1 /**
2  * @version 1.11 2018-05-01
3  * @author Cay Horstmann
4  */
5
6 public class EmployeeTest
7 {
8    public static void main(String[] args)
9    {
10      var staff = new Employee[3];
11
12      staff[0] = new Employee("Harry Hacker", 35000);
13      staff[1] = new Employee("Carl Cracker", 75000);
14      staff[2] = new Employee("Tony Tester", 38000);
15
16      for (Employee e : staff)
17         e.raiseSalary(5);
18      for (Employee e : staff)
19         e.print();
```

```
20    }
21 }
```

Listing 12.12

employee/Employee.java

Click here to view code image

```
 1 /**
 2  * @version 1.10 1999-11-13
 3  * @author Cay Horstmann
 4  */
 5
 6 public class Employee
 7 {
 8    private String name;
 9    private double salary;
10
11    public native void raiseSalary(double byPercent);
12
13    public Employee(String n, double s)
14    {
15       name = n;
16       salary = s;
17    }
18
19    public void print()
20    {
```

```
21       System.out.println(name + " " + salary);
22    }
23
24    static
25    {
26       System.loadLibrary("Employee");
27    }
28 }
```

Listing 12.13

employee/Employee.c

Click here to view code image

```
1  '/**
2     @version 1.10 1999-11-13
3     @author Cay Horstmann
4  */
5
6  #include "Employee.h"
7
8  #include <stdio.h>
9
10 JNIEXPORT void JNICALL Java_Employee_raiseSalary(
11       JNIEnv* env, jobject this_obj, jdouble byPercent)
12 {
13    /* get the class */
14    jclass class_Employee = (*env)->GetObjectClass(env, this_obj);
15
```

```
16    /* get the field ID */
17    jfieldID id_salary = (*env)->GetFieldID(env, class_Employee, "salary", "D");
18
19    /* get the field value */
20    jdouble salary = (*env)->GetDoubleField(env, this_obj, id_salary);
21
22    salary *= 1 + byPercent / 100;
23
24    /* set the field value */
25    (*env)->SetDoubleField(env, this_obj, id_salary, salary);
26 }
27
```

12.4.2 Accessing Static Fields

Accessing static fields is similar to accessing nonstatic fields. Use the

`GetStaticFieldID`

and

`GetStatic`*`Xxx`*`Field/SetStatic`*`Xxx`*`Field`

functions that work almost identically to their nonstatic counterparts, with two differences:

- Since you have no object, you must use

  `FindClass`

  instead of

  `GetObjectClass`

  to obtain the class reference.
- You have to supply the class, not the instance object, when accessing the field.

For example, here is how you can get a reference to

`System.out`

:

Click here to view code image

```
/* get the class */
jclass class_System = (*env)->FindClass(env, "java/lang/System");

/* get the field ID */
jfieldID id_out = (*env)->GetStaticFieldID(env, class_System, "out",
  "Ljava/io/PrintStream;");

/* get the field value */
jobject obj_out = (*env)->GetStaticObjectField(env, class_System, id_out);
```

Accessing Fields

`jfieldID GetFieldID(JNIEnv *env, jclass cl, const char name[], const char fieldSignature[])`

returns the identifier of a field in a class.

`Xxx GetXxxField(JNIEnv *env, jobject obj, jfieldID id)`

returns the value of a field. The field type

`Xxx`

is one of

`Object`

,

`Boolean`

,

`Byte`

,

`Char`

,

`Short`

,

`Int`

,

`Long`

,

`Float`

, or

`Double`

.

`void Set*Xxx*Field(JNIEnv *env, jobject obj, jfieldID id, *Xxx* value)`

sets a field to a new value. The field type

Xxx

is one of

`Object`

,

`Boolean`

,

`Byte`

,

`Char`

,

`Short`

,

`Int`

,

`Long`

,

`Float`

, or

`Double`

.

`jfieldID GetStaticFieldID(JNIEnv *env, jclass cl, const char name[], const char fieldSignature[])`

returns the identifier of a static field in a class.

`*Xxx* GetStatic*Xxx*Field(JNIEnv *env, jclass cl, jfieldID id)`

returns the value of a static field. The field type

`*Xxx*`

is one of

`Object`

,

`Boolean`

,

`Byte`

,
Char
,
Short
,
Int
,
Long
,
Float
, or
Double
.
void SetStatic*Xxx*Field(JNIEnv *env, jclass cl, jfieldID id, *Xxx* value)
sets a static field to a new value. The field type
Xxx
is one of
Object
,
Boolean
,
Byte
,
Char
,

`Short`
,
`Int`
,
`Long`
,
`Float`
, or
`Double`
.

12.5 Encoding Signatures

To access instance fields and call methods defined in the Java programming language, you need to learn the rules for “mangling” the names of data types and method signatures. (A method signature describes the parameters and return type of the method.) Here is the encoding scheme:

B	byte
C	char
D	double

F	float
I	int
J	long
L*classname*;	a class type
S	short
V	void
Z	boolean

To describe an array type, use a

```
[
```

. For example, an array of strings is

```
[Ljava/lang/String;
```

A

`float[][]`

is mangled into

```
[[F
```

For the complete signature of a method, list the parameter types inside a pair of parentheses and then list the return type. For example, a method receiving two integers and returning an integer is encoded as

```
(II)I
```

The `sprint` method in **Section 12.3**, "**String Parameters**," on p. 795 has a mangled signature of

Click here to view code image

```
(Ljava/lang/String;D)Ljava/lang/String;
```

That is, the method receives a `String`

and a

`double`

and returns a

`String`

.

Note that the semicolon at the end of the

`L`

expression is the terminator of the type expression, not a separator between parameters. For example, the constructor

Click here to view code image

```
Employee(java.lang.String, double, java.util.Date)
```

has a signature

Click here to view code image

```
"(Ljava/lang/String;DLjava/util/Date;)V"
```

Note that there is no separator between the

`D`

and

`Ljava/util/Date;`

. Also note that in this encoding scheme, you must use

`/`

instead of

`.`

to separate the package and class names. The

`V`

at the end denotes a return type of

`void`

. Even though you don't specify a return type for constructors in Java, you need to add a

`V`

to the virtual machine signature.



Tip

You can use the

`javap`

command with option

`-s`

to generate the method signatures from class files. For example, run

```
javap -s -private Employee
```

You will get the following output, displaying the signatures of all fields and methods:

Click here to view code image

```
Compiled from "Employee.java"
public class Employee extends java.lang.Object{
private java.lang.String name;
  Signature: Ljava/lang/String;
private double salary;
  Signature: D
public Employee(java.lang.String, double);
  Signature: (Ljava/lang/String;D)V
public native void raiseSalary(double);
  Signature: (D)V
public void print();
  Signature: ()V
static {};
  Signature: ()V
}
```



Note

There is no rationale whatsoever for forcing programmers to use this mangling scheme for signatures. The designers of the native calling mechanism could have just as easily written a function that reads signatures in the Java programming language style, such as

`void(int,java.lang.String)`

, and encodes them into whatever internal representation they prefer. Then again, using the mangled signatures lets you partake in the mystique of programming close to the virtual machine.

12.6 Calling Java Methods

Of course, Java programming language functions can call C functions—that is what native methods are for. Can we go the other way? Why would we want to do this anyway? It often happens that a native method needs to request a service from an object that was passed to it. I'll first show you how to do it for instance methods, then for static methods.

12.6.1 Instance Methods

As an example of calling a Java method from native code, let's enhance the

`Printf`

class and add a method that works similarly to the C function

`fprintf`

. That is, it should be able to print a string on an arbitrary

`PrintWriter`

object. Here is the definition of the method in Java:

Click here to view code image

```
class Printf3
{
  public native static void fprint(PrintWriter out, String s, double x);
```

```
  ...
}
```

We'll first assemble the string to be printed into a `String` object `str`, as in the `sprint` method that we already implemented. Then, from the C function that implements the native method, we'll call the `print` method of the `PrintWriter` class.

You can call any Java method from C by using the function call

Click here to view code image

```
(*env)->Call
Xxx
Method(env,
implicit parameter
,
methodID
,
```

```
    explicit parameters
    )
```

Replace

```
Xxx
```

with

```
Void
```

,

```
Int
```

,

```
Object
```

, and so on, depending on the return type of the method. Just as you need a

```
fieldID
```

to access a field of an object, you need a method ID to call a method. To obtain a method ID, call the JNI function

```
GetMethodID
```

and supply the class, the name of the method, and the method signature.

In our example, we want to obtain the ID of the

```
print
```

method of the

```
PrintWriter
```

class. The

```
PrintWriter
```

class has several overloaded methods called

```
print
```

. For that reason, you must also supply a string describing the parameters and the return value of the specific function that you want to use. For example, we want to use

`void print(java.lang.String)`

. As described in the preceding section, we must now "mangle" the signature into the string

`"(Ljava/lang/String;)V"`

.

Here is the complete code to make the method call:

Click here to view code image

```
/* get the class of the implicit parameter */
class_PrintWriter = (*env)->GetObjectClass(env, out);

/* get the method ID */
id_print = (*env)->GetMethodID(env, class_PrintWriter, "print", "(Ljava/lang/String;)V");

/* call the method */
(*env)->CallVoidMethod(env, out, id_print, str);
```

Listings 12.14 and 12.15 show the Java code for a test program and the

`Printf3`

class. Listing 12.16 contains the C code for the native

`fprint`

method.



Note

The numerical method IDs and field IDs are conceptually similar to

`Method`

and

`Field`

objects in the reflection API. You can convert between them with the following functions:

Click here to view code image

```
jobject ToReflectedMethod(JNIEnv* env, jclass class, jmethodID methodID);
  // returns Method object
methodID FromReflectedMethod(JNIEnv* env, jobject method);
jobject ToReflectedField(JNIEnv* env, jclass class, jfieldID fieldID);
  // returns Field object
fieldID FromReflectedField(JNIEnv* env, jobject field);
```

12.6.2 Static Methods

Calling static methods from native methods is similar to calling instance methods. There are two differences:

Use the

`GetStaticMethodID`

and

CallStatic*Xxx*Method

functions

Supply a class object, not an implicit parameter object, when invoking the method

As an example of this, let's make the call to the static method

Click here to view code image

```
System.getProperty("java.class.path")
```

from a native method. The return value of this call is a string that gives the current class path.

First, we have to find the class to use. As we have no object of the class

```
System
```

readily available, we use

```
FindClass
```

rather than

```
GetObjectClass
```

.

Click here to view code image

```
jclass class_System = (*env)->FindClass(env, "java/lang/System");
```

Next, we need the ID of the static

`getProperty`

method. The encoded signature of that method is

Click here to view code image

```
"(Ljava/lang/String;)Ljava/lang/String;"
```

because both the parameter and the return value are strings. Hence, we obtain the method ID as follows:

Click here to view code image

```
jmethodID id_getProperty = (*env)->GetStaticMethodID(env, class_System, "getProperty",
  "(Ljava/lang/String;)Ljava/lang/String;");
```

Finally, we can make the call. Note that the class object is passed to the

`CallStaticObjectMethod`

function.

Click here to view code image

```
jobject obj_ret = (*env)->CallStaticObjectMethod(env, class_System, id_getProperty,
```

```
    (*env)->NewStringUTF(env, "java.class.path"));
```

The return value of this method is of type

`jobject`

. If we want to manipulate it as a string, we must cast it to

`jstring`

:

Click here to view code image

```
    jstring str_ret = (jstring) obj_ret;
```



C++ Note

In C, the types

`jstring`

and

`jclass`

, as well as the array types to be introduced later, are all type-equivalent to

`jobject`

. The cast of the preceding example is therefore not strictly necessary in C. But in C++, these types are defined as pointers to “dummy classes” that have the correct inheritance hierarchy. For example, assigning a

`jstring`

to a

`jobject`

is legal without a cast in C++, but an assignment from a

`jobject`

to a

`jstring`

requires a cast.

12.6.3 Constructors

A native method can create a new Java object by invoking its constructor. Invoke the constructor by calling the

`NewObject`

function.

Click here to view code image

```
jobject obj_new = (*env)->NewObject(env,
class
,
methodID
,
construction parameters
);
```

You can obtain the method ID needed for this call from the

`GetMethodID`

function by specifying the method name as

`"<init>"`

and the encoded signature of the constructor (with return type

`void`

). For example, here is how a native method can create a

`FileOutputStream`

object:

Click here to view code image

```
const char[] fileName = ". . .";
jstring str_fileName = (*env)->NewStringUTF(env, fileName);
jclass class_FileOutputStream = (*env)->FindClass(env, "java/io/FileOutputStream");
jmethodID id_FileOutputStream
   = (*env)->GetMethodID(env, class_FileOutputStream, "<init>", "(Ljava/lang/String;)V");
jobject obj_stream
   = (*env)->NewObject(env, class_FileOutputStream, id_FileOutputStream, str_fileName);
```

Note that the signature of the constructor takes a parameter of type

`java.lang.String`

and has a return type of

`void`

.

12.6.4 Alternative Method Invocations

Several variants of the JNI functions can be used to call a Java method from native code. These are not as important as the functions that we already discussed, but they are occasionally useful.

The

`CallNonvirtual`*`Xxx`*`Method`

functions receive an implicit argument, a method ID, a class object (which must correspond to a superclass of the implicit argument), and explicit arguments. The function calls the version of the method in the specified class, bypassing the normal dynamic dispatch mechanism.

All call functions have versions with suffixes "A" and "V" that receive the explicit parameters in an array or a

`va_list`

(as defined in the C header

`stdarg.h`

).

Listing 12.14

printf3/Printf3Test.java

Click here to view code image

```
1 import java.io.*;
2
3 /**
4  * @version 1.11 2018-05-01
5  * @author Cay Horstmann
6  */
```

```
 7 class Printf3Test
 8 {
 9    public static void main(String[] args)
10    {
11       double price = 44.95;
12       double tax = 7.75;
13       double amountDue = price * (1 + tax / 100);
14       var out = new PrintWriter(System.out);
15       Printf3.fprint(out, "Amount due = %8.2f\n", amountDue);
16       out.flush();
17    }
18 }
```

Listing 12.15

printf3/Printf3.java

Click here to view code image

```
 1 import java.io.*;
 2
 3 /**
 4  * @version 1.10 1997-07-01
 5  * @author Cay Horstmann
 6  */
 7 class Printf3
 8 {
 9    public static native void fprint(PrintWriter out, String format, double x);
10
11    static
```

```
12    {
13       System.loadLibrary("Printf3");
14    }
15 }
```

Listing 12.16

printf3/Printf3.c

Click here to view code image

```
 1 /**
 2    @version 1.10 1997-07-01
 3    @author Cay Horstmann
 4 */
 5
 6 #include "Printf3.h"
 7 #include <string.h>
 8 #include <stdlib.h>
 9 #include <float.h>
10
11 /**
12    @param format a string containing a printf format specifier
13    (such as "%8.2f"). Substrings "%%" are skipped.
14    @return a pointer to the format specifier (skipping the '%')
15    or NULL if there wasn't a unique format specifier
16 */
17 char* find_format(const char format[])
18 {
19    char* p;
```

```
20    char* q;
21
22    p = strchr(format, '%');
23    while (p != NULL && *(p + 1) == '%') /* skip %% */
24       p = strchr(p + 2, '%');
25    if (p == NULL) return NULL;
26    /* now check that % is unique */
27    p++;
28    q = strchr(p, '%');
29    while (q != NULL && *(q + 1) == '%') /* skip %% */
30       q = strchr(q + 2, '%');
31    if (q != NULL) return NULL; /* % not unique */
32    q = p + strspn(p, " -0+#"); /* skip past flags */
33    q += strspn(q, "0123456789"); /* skip past field width */
34    if (*q == '.') { q++; q += strspn(q, "0123456789"); }
35       /* skip past precision */
36    if (strchr("eEfFgG", *q) == NULL) return NULL;
37       /* not a floating-point format */
38    return p;
39 }
40
41 JNIEXPORT void JNICALL Java_Printf3_fprint(JNIEnv* env, jclass cl,
42       jobject out, jstring format, jdouble x)
43 {
44    const char* cformat;
45    char* fmt;
46    jstring str;
47    jclass class_PrintWriter;
48    jmethodID id_print;
49
50    cformat = (*env)->GetStringUTFChars(env, format, NULL);
51    fmt = find_format(cformat);
```

```
52    if (fmt == NULL)
53       str = format;
54    else
55    {
56       char* cstr;
57       int width = atoi(fmt);
58       if (width == 0) width = DBL_DIG + 10;
59       cstr = (char*) malloc(strlen(cformat) + width);
60       sprintf(cstr, cformat, x);
61       str = (*env)->NewStringUTF(env, cstr);
62       free(cstr);
63    }
64    (*env)->ReleaseStringUTFChars(env, format, cformat);
65
66    /* now call out.print(str) */
67
68    /* get the class */
69    class_PrintWriter = (*env)->GetObjectClass(env, out);
70
71    /* get the method ID */
72    id_print = (*env)->GetMethodID(env, class_PrintWriter, "print", "(Ljava/lang/String;)
V");
73
74    /* call the method */
75    (*env)->CallVoidMethod(env, out, id_print, str);
76 }
```

Executing Java Methods

```
jmethodID GetMethodID(JNIEnv *env, jclass cl, const char name[], const char methodSignature[])
```

returns the identifier of a method in a class.

```
Xxx CallXxxMethod(JNIEnv *env, jobject obj, jmethodID id, args)
```

```
Xxx CallXxxMethodA(JNIEnv *env, jobject obj, jmethodID id, jvalue args[])
```

```
Xxx CallXxxMethodV(JNIEnv *env, jobject obj, jmethodID id, va_list args)
```

call a method. The return type

```
Xxx
```

is one of

```
Object
```

,

```
Boolean
```

,

```
Byte
```

,

```
Char
```

,

```
Short
```

,

```
Int
```

,

```
Long
```

,

```
Float
```

, or

`Double`

. The first function has a variable number of arguments—simply append the method parameters after the method ID. The second function receives the method arguments in an array of

`jvalue`

, where

`jvalue`

is a union defined as

```
typedef union jvalue
{
  jboolean z;
  jbyte b;
  jchar c;
  jshort s;
  jint i;
  jlong j;
  jfloat f;
  jdouble d;
  jobject l;
} jvalue;
```

The third function receives the method parameters in a

`va_list`

, as defined in the C header

`stdarg.h`

.

Xxx CallNonvirtual*Xxx*Method(JNIEnv *env, jobject obj, jclass cl, jmethodID id, args)

Xxx CallNonvirtual*Xxx*MethodA(JNIEnv *env, jobject obj, jclass cl, jmethodID id, jvalue args[])

Xxx CallNonvirtual*Xxx*MethodV(JNIEnv *env, jobject obj, jclass cl, jmethodID id, va_list args)

call a method, bypassing dynamic dispatch. The return type

Xxx

is one of

Object

,

Boolean

,

Byte

,

Char

,

Short

,

Int

,

Long

,

Float

, or

Double

. The first function has a variable number of arguments—simply append the method parameters after the method ID. The second function receives the method arguments in an array of

jvalue

. The third function receives the method parameters in a

va_list

, as defined in the C header

stdarg.h

.

jmethodID GetStaticMethodID(JNIEnv *env, jclass cl, const char name[], const char methodSignature[])

returns the identifier of a static method in a class.

Xxx CallStatic*Xxx*Method(JNIEnv *env, jclass cl, jmethodID id, args)

Xxx CallStatic*Xxx*MethodA(JNIEnv *env, jclass cl, jmethodID id, jvalue args[])

Xxx CallStatic*Xxx*MethodV(JNIEnv *env, jclass cl, jmethodID id, va_list args)

call a static method. The return type

Xxx

is one of

Object

,

Boolean

,

Byte

,

Char

,

Short

,

Int

,

`Long`

,

`Float`

, or

`Double`

. The first function has a variable number of arguments—simply append the method parameters after the method ID. The second function receives the method arguments in an array of

`jvalue`

. The third function receives the method parameters in a

`va_list`

, as defined in the C header

`stdarg.h`

.

```
jobject NewObject(JNIEnv *env, jclass cl, jmethodID id, args)
jobject NewObjectA(JNIEnv *env, jclass cl, jmethodID id, jvalue args[])
jobject NewObjectV(JNIEnv *env, jclass cl, jmethodID id, va_list args)
```

call a constructor. The method ID is obtained from

`GetMethodID`

with a method name of

`"<init>"`

and a return type of

`void`

. The first function has a variable number of arguments—simply append the method parameters after the method ID. The second function receives the method arguments in an array of

`jvalue`

. The third function receives the method parameters in a

`va_list`

, as defined in the C header

`stdarg.h`

.

12.7 Accessing Array Elements

All array types of the Java programming language have corresponding C types, as shown in **Table 12.2**.

Table 12.2 Correspondence between Java Array Types and C Types

Java Type	C Type
`boolean[]`	`jbooleanArray`
`byte[]`	`jbyteArray`
`char[]`	`jcharArray`
`int[]`	`jintArray`

Java Type	C Type
short[]	jshortArray
long[]	jlongArray
float[]	jfloatArray
double[]	jdoubleArray
Object[]	jobjectArray



C++ Note

In C, all these array types are actually type synonyms of `jobject`. In C++, however, they are arranged in the inheritance hierarchy shown in **Figure 12.3**. The type `jarray` denotes a generic array.

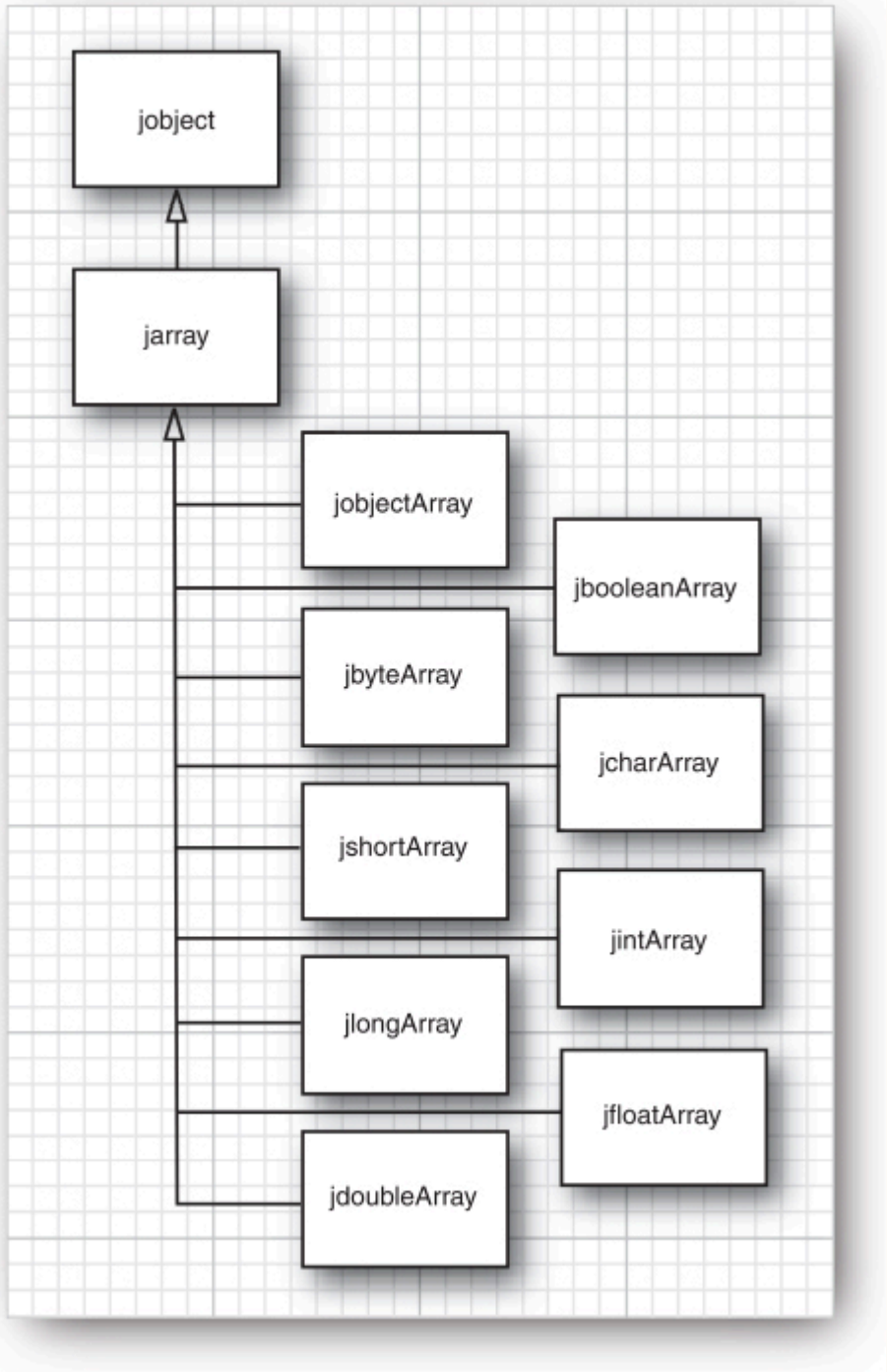

jobject
jarray
jobjectArray
jbooleanArray
jbyteArray
jcharArray
jshortArray
jintArray
jlongArray
jfloatArray
jdoubleArray

Figure 12.3 Inheritance hierarchy of array types

The `GetArrayLength` function returns the length of an array.

Click here to view code image

```
jarray array = . . .;
jsize length = (*env)->GetArrayLength(env, array);
```

How you access elements in an array depends on whether the array stores objects or values of a primitive type (`bool`, `char`, or a numeric type). To access elements in an object array, use the `GetObjectArrayElement` and `SetObjectArrayElement` methods.

Click here to view code image

```
jobjectArray array = . . .;
int i, j;
jobject x = (*env)->GetObjectArrayElement(env, array, i);
(*env)->SetObjectArrayElement(env, array, j, x);
```

Although simple, this approach is also clearly inefficient; you want to be able to access array elements directly, especially when doing vector and matrix computations.

The `Get*Xxx*ArrayElements` function returns a C pointer to the starting element of an array. As with ordinary strings, you must remember to call the corresponding `Release*Xxx*ArrayElements` function to tell the virtual machine when you no longer need that pointer. Here, the type `*Xxx*` must be a primitive type—that is, not `Object`. You can then read and write the array elements directly. However, since the pointer *might point to a copy*, any changes that you make are guaranteed to be reflected in the original array only after you call the corresponding `Release*Xxx*ArrayElements` function!



Note

You can find out if an array is a copy by passing a pointer to a `jboolean` variable as the third parameter to a `Get*Xxx*ArrayElements` method. The variable is filled with `JNI_TRUE` if the array is a copy. If you aren't interested in that information, just pass a `NULL` pointer.

Here is an example that multiplies all elements in an array of `double` values by a constant. We obtain a C pointer `a` into the Java array and then access individual elements as `a[i]`.

Click here to view code image

```
jdoubleArray array_a = . . .;
double scaleFactor = . . .;
double* a = (*env)->GetDoubleArrayElements(env, array_a, NULL);
```

```
for (i = 0; i < (*env)->GetArrayLength(env, array_a); i++)
  a[i] = a[i] * scaleFactor;
(*env)->ReleaseDoubleArrayElements(env, array_a, a, 0);
```

Whether the virtual machine actually copies the array depends on how it allocates arrays and does its garbage collection. Some "copying" garbage collectors routinely move objects around and update object references. That strategy is not compatible with "pinning" an array to a particular location, because the collector cannot update the pointer values in native code.

 **Note**

In the Oracle JVM implementation,
`boolean`
arrays are represented as packed arrays of 32-bit words. The
`GetBooleanArrayElements`
method copies them into unpacked arrays of
`jboolean`
values.

To access just a few elements of a large array, use the
`Get*Xxx*ArrayRegion`
and
`Set*Xxx*ArrayRegion`
methods that copy a range of elements from the Java array into a C array and back.

You can create new Java arrays in native methods with the

New*Xxx*Array

function. To create a new array of objects, specify the length, the type of the array elements, and an initial element for all entries (typically,

NULL

). Here is an example:

Click here to view code image

```
jclass class_Employee = (*env)->FindClass(env, "Employee");
jobjectArray array_e = (*env)->NewObjectArray(env, 100, class_Employee, NULL);
```

Arrays of primitive types are simpler: Just supply the length of the array.

Click here to view code image

```
jdoubleArray array_d = (*env)->NewDoubleArray(env, 100);
```

The array is then filled with zeroes.



Note

The following methods are used for working with "direct buffers":

Click here to view code image

```
jobject NewDirectByteBuffer(JNIEnv* env, void* address, jlong capacity)
void* GetDirectBufferAddress(JNIEnv* env, jobject buf)
jlong GetDirectBufferCapacity(JNIEnv* env, jobject buf)
```

Direct buffers are used in the

`java.nio`

package to support more efficient input/output operations and to minimize the copying of data between native and Java arrays.

Manipulating Java Arrays

`jsize GetArrayLength(JNIEnv *env, jarray array)`

returns the number of elements in the array.

`jobject GetObjectArrayElement(JNIEnv *env, jobjectArray array, jsize index)`

returns the value of an array element.

`void SetObjectArrayElement(JNIEnv *env, jobjectArray array, jsize index, jobject value)`

sets an array element to a new value.

Xxx`* Get`*Xxx*`ArrayElements(JNIEnv *env, jarray array, jboolean* isCopy)`

yields a C pointer to the elements of a Java array. The field type

Xxx

is one of

`Boolean`
,
`Byte`
,
`Char`
,
`Short`
,
`Int`
,
`Long`
,
`Float`
, or
`Double`
. The pointer must be passed to
`Release`*`Xxx`*`ArrayElements`
when it is no longer needed.
`isCopy`
is either
`NULL`
or points to a
`jboolean`
that is filled with
`JNI_TRUE`

if a copy is made, with

JNI_FALSE

otherwise.

void Release*Xxx*ArrayElements(JNIEnv *env, jarray array, *Xxx* elems[], jint mode)

notifies the virtual machine that a pointer obtained by

Get*Xxx*ArrayElements

is no longer needed.

mode

is one of

0

(free the

elems

buffer after updating the array elements),

JNI_COMMIT

(do not free the

elems

buffer after updating the array elements), or

JNI_ABORT

(free the

elems

buffer without updating the array elements).

void Get*Xxx*ArrayRegion(JNIEnv *env, jarray array, jint start, jint length, *Xxx* elems[])

copies elements from a Java array to a C array. The field type

Xxx

is one of

`Boolean`

,

`Byte`

,

`Char`

,

`Short`

,

`Int`

,

`Long`

,

`Float`

, or

`Double`

.

`void Set*Xxx*ArrayRegion(JNIEnv *env, jarray array, jint start, jint length, *Xxx* elems[])`

copies elements from a C array to a Java array. The field type

Xxx

is one of

`Boolean`

,

`Byte`

,

`Char`

,
`Short`
,
`Int`
,
`Long`
,
`Float`
, or
`Double`
.

12.8 Handling Errors

Native methods are a significant security risk to Java programs. The C runtime system has no protection against array bounds errors, indirection through bad pointers, and so on. It is particularly important that programmers of native methods handle all error conditions to preserve the integrity of the Java platform. In particular, when your native method diagnoses a problem it cannot handle, it should report this problem to the Java virtual machine.

Normally, you would throw an exception in this situation. However, C has no exceptions. Instead, you must call the
`Throw`
or
`ThrowNew`
function to create a new exception object. When the native method exits, the Java virtual machine throws that exception.

To use the

`Throw`

function, call

`NewObject`

to create an object of a subtype of

`Throwable`

. For example, here we allocate an

`EOFException`

object and throw it:

Click here to view code image

```
jclass class_EOFException = (*env)->FindClass(env, "java/io/EOFException");
jmethodID id_EOFException = (*env)->GetMethodID(env, class_EOFException, "<init>", "()V");
   /* ID of no-argument constructor */
jthrowable obj_exc = (*env)->NewObject(env, class_EOFException, id_EOFException);
(*env)->Throw(env, obj_exc);
```

It is usually more convenient to call

`ThrowNew`

, which constructs an exception object, given a class and a “modified UTF-8” byte sequence.

Click here to view code image

```
(*env)->ThrowNew(env, (*env)->FindClass(env, "java/io/EOFException"),
  "Unexpected end of file");
```

Both `Throw` and `ThrowNew` merely *post* the exception; they do not interrupt the control flow of the native method. Only when the method returns does the Java virtual machine throw the exception. Therefore, every call to `Throw` and `ThrowNew` should be immediately followed by a `return` statement.



C++ Note

If you implement native methods in C++, you cannot throw a Java exception object in your C++ code. In a C++ binding, it would be possible to implement a translation between exceptions in C++ and Java; however, this is not currently done. Use `Throw` or

ThrowNew

to throw a Java exception in a native C++ method, and make sure your native methods throw no C++ exceptions.

Normally, native code need not be concerned with catching Java exceptions. However, when a native method calls a Java method, that method might throw an exception. Moreover, a number of the JNI functions throw exceptions as well. For example,

SetObjectArrayElement

throws an

ArrayIndexOutOfBoundsException

if the index is out of bounds, and an

ArrayStoreException

if the class of the stored object is not a subclass of the element class of the array. In situations like these, a native method should call the

ExceptionOccurred

method to determine whether an exception has been thrown. The call

Click here to view code image

```
jthrowable obj_exc = (*env)->ExceptionOccurred(env);
```

returns

NULL

if no exception is pending, or a reference to the current exception object. If you just want to check whether an exception has been thrown, without obtaining a reference to the exception object, use

Click here to view code image

```
jboolean occurred = (*env)->ExceptionCheck(env);
```

Normally, a native method should simply return when an exception has occurred so that the virtual machine can propagate it to the Java code. However, a native method *may* analyze the exception object to determine if it can handle the exception. If it can, then the function

```
(*env)->ExceptionClear(env);
```

must be called to turn off the exception.

In our next example, we implement the `fprint` native method with all the paranoia appropriate for a native method. Here are the exceptions that we throw:

- A `NullPointerException` if the format string is `NULL`
- An `IllegalArgumentException` if the format string doesn't contain a

`%`

specifier that is appropriate for printing a

`double`

An

`OutOfMemoryError`

if the call to

`malloc`

fails

Finally, to demonstrate how to check for an exception when calling a Java method from a native method, we send the string to the stream, a character at a time, and call

`ExceptionOccurred`

after each call. **Listing 12.17** shows the code for the native method, and **Listing 12.18** shows the definition of the class containing the native method. Notice that the native method does not immediately terminate when an exception occurs in the call to

`PrintWriter.print`

—it first frees the

`cstr`

buffer. When the native method returns, the virtual machine again raises the exception. The test program in **Listing 12.19** demonstrates how the native method throws an exception when the formatting string is not valid.

Listing 12.17

printf4/Printf4.c

Click here to view code image

```
/**
   @version 1.10 1997-07-01
   @author Cay Horstmann
*/

#include "Printf4.h"
#include <string.h>
#include <stdlib.h>
#include <float.h>

/**
   @param format a string containing a printf format specifier
   (such as "%8.2f"). Substrings "%%" are skipped.
   @return a pointer to the format specifier (skipping the '%')
   or NULL if there wasn't a unique format specifier
*/
char* find_format(const char format[])
{
   char* p;
   char* q;

   p = strchr(format, '%');
   while (p != NULL && *(p + 1) == '%') /* skip %% */
      p = strchr(p + 2, '%');
   if (p == NULL) return NULL;
   /* now check that % is unique */
   p++;
   q = strchr(p, '%');
   while (q != NULL && *(q + 1) == '%') /* skip %% */
      q = strchr(q + 2, '%');
   if (q != NULL) return NULL; /* % not unique */
```

```
   q = p + strspn(p, " -0+#"); /* skip past flags */
   q += strspn(q, "0123456789"); /* skip past field width */
   if (*q == '.') { q++; q += strspn(q, "0123456789"); }
      /* skip past precision */
   if (strchr("eEfFgG", *q) == NULL) return NULL;
      /* not a floating-point format */
   return p;
}

JNIEXPORT void JNICALL Java_Printf4_fprint(JNIEnv* env, jclass cl,
      jobject out, jstring format, jdouble x)
{
   const char* cformat;
   char* fmt;
   jclass class_PrintWriter;
   jmethodID id_print;
   char* cstr;
   int width;
   int i;

   if (format == NULL)
   {
      (*env)->ThrowNew(env,
         (*env)->FindClass(env,
         "java/lang/NullPointerException"),
         "Printf4.fprint: format is null");
      return;
   }

   cformat = (*env)->GetStringUTFChars(env, format, NULL);
   fmt = find_format(cformat);

```

```
64    if (fmt == NULL)
65    {
66       (*env)->ThrowNew(env,
67          (*env)->FindClass(env,
68          "java/lang/IllegalArgumentException"),
69          "Printf4.fprint: format is invalid");
70       return;
71    }
72
73    width = atoi(fmt);
74    if (width == 0) width = DBL_DIG + 10;
75    cstr = (char*)malloc(strlen(cformat) + width);
76
77    if (cstr == NULL)
78    {
79       (*env)->ThrowNew(env,
80          (*env)->FindClass(env, "java/lang/OutOfMemoryError"),
81          "Printf4.fprint: malloc failed");
82       return;
83    }
84
85    sprintf(cstr, cformat, x);
86
87    (*env)->ReleaseStringUTFChars(env, format, cformat);
88
89    /* now call ps.print(str) */
90
91    /* get the class */
92    class_PrintWriter = (*env)->GetObjectClass(env, out);
93
94    /* get the method ID */
95    id_print = (*env)->GetMethodID(env, class_PrintWriter, "print", "(C)V");
96
```

```
   /* call the method */
   for (i = 0; cstr[i] != 0 && !(*env)->ExceptionOccurred(env); i++)
      (*env)->CallVoidMethod(env, out, id_print, cstr[i]);

   free(cstr);
}
```

Listing 12.18

printf4/Printf4.java

Click here to view code image

```
import java.io.*;

/**
 * @version 1.10 1997-07-01
 * @author Cay Horstmann
 */
class Printf4
{
   public static native void fprint(PrintWriter ps, String format, double x);

   static
   {
      System.loadLibrary("Printf4");
   }
}
```

Listing 12.19

printf4/Printf4Test.java

Click here to view code image

```
 1 import java.io.*;
 2
 3 /**
 4  * @version 1.11 2018-05-01
 5  * @author Cay Horstmann
 6  */
 7 class Printf4Test
 8 {
 9    public static void main(String[] args)
10    {
11       double price = 44.95;
12       double tax = 7.75;
13       double amountDue = price * (1 + tax / 100);
14       var out = new PrintWriter(System.out);
15       /* This call will throw an exception--note the %% */
16       Printf4.fprint(out, "Amount due = %%8.2f\n", amountDue);
17       out.flush();
18    }
19 }
```

`jint Throw(JNIEnv *env, jthrowable obj)`

prepares an exception to be thrown upon exiting from the native code. Returns

`0`

on success, a negative value on failure.

`jint ThrowNew(JNIEnv *env, jclass cl, const char msg[])`

prepares an exception of type

`cl`

to be thrown upon exiting from the native code. Returns

`0`

on success, a negative value on failure.

`msg`

is a “modified UTF-8” byte sequence denoting the

`String`

construction argument of the exception object.

`jthrowable ExceptionOccurred(JNIEnv *env)`

returns the exception object if an exception is pending, or

`NULL`

otherwise.

`jboolean ExceptionCheck(JNIEnv *env)`

returns

`true`

if an exception is pending.

`void ExceptionClear(JNIEnv *env)`

clears any pending exceptions.

12.9 Using the Invocation API

Up to now, we have considered programs in the Java programming language that made a few C calls, presumably because C was faster or allowed access to functionality inaccessible from the Java platform. Suppose you are in the opposite situation. You have a C or C++ program and would like to make calls to Java code. The *invocation API* enables you to embed the Java virtual machine into a C or C++ program. Here is the minimal code that you need to initialize a virtual machine:

Click here to view code image

```
JavaVMOption options[1];
JavaVMInitArgs vm_args;
JavaVM *jvm;
JNIEnv *env;

options[0].optionString = "-Djava.class.path=.";

memset(&vm_args, 0, sizeof(vm_args));
vm_args.version = JNI_VERSION_1_2;
vm_args.nOptions = 1;
vm_args.options = options;

JNI_CreateJavaVM(&jvm, (void**) &env, &vm_args);
```

The call to

`JNI_CreateJavaVM`

creates the virtual machine and fills in a pointer

`jvm`

to the virtual machine and a pointer

`env`

to the execution environment.

You can supply any number of options to the virtual machine. Simply increase the size of the

`options`

array and the value of

`vm_args.nOptions`

. For example,

Click here to view code image

```
options[i].optionString = "-Djava.compiler=NONE";
```

deactivates the just-in-time compiler.



Tip

When you run into trouble and your program crashes, refuses to initialize the JVM, or can't load your classes, turn on the JNI debugging mode. Set an option to

Click here to view code image

```
options[i].optionString = "-verbose:jni";
```

You will see a flurry of messages that indicate the progress in initializing the JVM. If you don't see your classes loaded, check both your path and class path settings.

Once you have set up the virtual machine, you can call Java methods as described in the preceding sections. Simply use the `env` pointer in the usual way.

You'll need the `jvm` pointer only to call other functions in the invocation API. Currently, there are only four such functions. The most important one is the function to terminate the virtual machine:

```
(*jvm)->DestroyJavaVM(jvm);
```

Unfortunately, under Windows, it has become difficult to dynamically link to the `JNI_CreateJavaVM` function in the

jre/bin/client/jvm.dll

library, due to the changed linking rules in Vista and Oracle's reliance on an older C runtime library. Our sample program overcomes this problem by loading the library manually. This is the same approach used by the

java

program—see the file

launcher/java_md.c

in the

src.jar

file that is a part of the JDK.

The C program in **Listing 12.20** sets up a virtual machine and calls the

main

method of the

Welcome

class, which was discussed in Volume I, **Chapter 2**. (Make sure to compile the

Welcome.java

file before starting the invocation test program.)

Listing 12.20

invocation/InvocationTest.c

Click here to view code image

```
1 /**
2    @version 1.20 2007-10-26
3    @author Cay Horstmann
```

```
 4 */
 5
 6 #include <jni.h>
 7 #include <stdlib.h>
 8
 9 #ifdef _WINDOWS
10
11 #include <windows.h>
12 static HINSTANCE loadJVMLibrary(void);
13 typedef jint (JNICALL *CreateJavaVM_t)(JavaVM **, void **, JavaVMInitArgs *);
14
15 #endif
16
17 int main()
18 {
19    JavaVMOption options[2];
20    JavaVMInitArgs vm_args;
21    JavaVM *jvm;
22    JNIEnv *env;
23    long status;
24
25    jclass class_Welcome;
26    jclass class_String;
27    jobjectArray args;
28    jmethodID id_main;
29
30 #ifdef _WINDOWS
31    HINSTANCE hjvmlib;
32    CreateJavaVM_t createJavaVM;
33 #endif
34
35    options[0].optionString = "-Djava.class.path=.";
36
```

```
 37    memset(&vm_args, 0, sizeof(vm_args));
 38    vm_args.version = JNI_VERSION_1_2;
 39    vm_args.nOptions = 1;
 40    vm_args.options = options;
 41
 42 #ifdef _WINDOWS
 43    hjvmlib = loadJVMLibrary();
 44    createJavaVM = (CreateJavaVM_t) GetProcAddress(hjvmlib, "JNI_CreateJavaVM");
 45    status = (*createJavaVM)(&jvm, (void **) &env, &vm_args);
 46 #else
 47    status = JNI_CreateJavaVM(&jvm, (void **) &env, &vm_args);
 48 #endif
 49
 50    if (status == JNI_ERR)
 51    {
 52       fprintf(stderr, "Error creating VM\n");
 53       return 1;
 54    }
 55
 56    class_Welcome = (*env)->FindClass(env, "Welcome");
 57    id_main = (*env)->GetStaticMethodID(env, class_Welcome, "main", "([Ljava/lang/String;)
V");
 58
 59    class_String = (*env)->FindClass(env, "java/lang/String");
 60    args = (*env)->NewObjectArray(env, 0, class_String, NULL);
 61    (*env)->CallStaticVoidMethod(env, class_Welcome, id_main, args);
 62
 63    (*jvm)->DestroyJavaVM(jvm);
 64
 65    return 0;
 66 }
 67
 68 #ifdef _WINDOWS
```

```
 69
 70 static int GetStringFromRegistry(HKEY key, const char *name, char *buf, jint bufsize)
 71 {
 72    DWORD type, size;
 73
 74    return RegQueryValueEx(key, name, 0, &type, 0, &size) == 0
 75       && type == REG_SZ
 76       && size < (unsigned int) bufsize
 77       && RegQueryValueEx(key, name, 0, 0, buf, &size) == 0;
 78 }
 79
 80 static void GetPublicJREHome(char *buf, jint bufsize)
 81 {
 82    HKEY key, subkey;
 83    char version[MAX_PATH];
 84
 85    /* Find the current version of the JRE */
 86    char *JRE_KEY = "Software\\JavaSoft\\Java Runtime Environment";
 87    if (RegOpenKeyEx(HKEY_LOCAL_MACHINE, JRE_KEY, 0, KEY_READ, &key) != 0)
 88    {
 89       fprintf(stderr, "Error opening registry key '%s'\n", JRE_KEY);
 90       exit(1);
 91    }
 92
 93    if (!GetStringFromRegistry(key, "CurrentVersion", version, sizeof(version)))
 94    {
 95       fprintf(stderr, "Failed reading value of registry key:\n\t%s\\CurrentVersion\n",
 96          JRE_KEY);
 97       RegCloseKey(key);
 98       exit(1);
 99    }
100
101    /* Find directory where the current version is installed. */
```

```
102    if (RegOpenKeyEx(key, version, 0, KEY_READ, &subkey) != 0)
103    {
104       fprintf(stderr, "Error opening registry key '%s\\%s'\n", JRE_KEY, version);
105       RegCloseKey(key);
106       exit(1);
107    }
108
109    if (!GetStringFromRegistry(subkey, "JavaHome", buf, bufsize))
110    {
111       fprintf(stderr, "Failed reading value of registry key:\n\t%s\\%s\\JavaHome\n",
112          JRE_KEY, version);
113       RegCloseKey(key);
114       RegCloseKey(subkey);
115       exit(1);
116    }
117
118    RegCloseKey(key);
119    RegCloseKey(subkey);
120 }
121
122 static HINSTANCE loadJVMLibrary(void)
123 {
124    HINSTANCE h1, h2;
125    char msvcdll[MAX_PATH];
126    char javadll[MAX_PATH];
127    GetPublicJREHome(msvcdll, MAX_PATH);
128    strcpy(javadll, msvcdll);
129    strncat(msvcdll, "\\bin\\msvcr71.dll", MAX_PATH - strlen(msvcdll));
130    msvcdll[MAX_PATH - 1] = '\0';
131    strncat(javadll, "\\bin\\client\\jvm.dll", MAX_PATH - strlen(javadll));
132    javadll[MAX_PATH - 1] = '\0';
133
134    h1 = LoadLibrary(msvcdll);
```

```
135    if (h1 == NULL)
136    {
137       fprintf(stderr, "Can't load library msvcr71.dll\n");
138       exit(1);
139    }
140
141    h2 = LoadLibrary(javadll);
142    if (h2 == NULL)
143    {
144       fprintf(stderr, "Can't load library jvm.dll\n");
145       exit(1);
146    }
147    return h2;
148 }
149
150 #endif
```

To compile this program under Linux, use

Click here to view code image

```
gcc -I
jdk
/include -I
jdk
/include/linux -o InvocationTest \
  -L
jdk
/jre/lib/i386/client -ljvm InvocationTest.c
```

When compiling in Windows with the Microsoft compiler, use the command line

Click here to view code image

```
cl -D_WINDOWS -I
jdk
\include -I
jdk
\include\win32 InvocationTest.c \

jdk
\lib\jvm.lib advapi32.lib
```

You will need to make sure that the

```
INCLUDE
```

and

```
LIB
```

environment variables include the paths to the Windows API header and library files.

Using Cygwin, compile with

Click here to view code image

```
gcc -D_WINDOWS -mno-cygwin -I
jdk
\include -I
```

```
jdk
\include\win32 -D_int64="long long" \
  -I c:\cygwin\usr\include\w32api -o InvocationTest
```

Before you run the program under Linux/UNIX, make sure that the

```
LD_LIBRARY_PATH
```

contains the directories for the shared libraries. For example, if you use the

```
bash
```

shell on Linux, issue the following command:

Click here to view code image

```
export LD_LIBRARY_PATH=
jdk
/jre/lib/i386/client:$LD_LIBRARY_PATH
```

Invocation API Functions

```
jint JNI_CreateJavaVM(JavaVM** p_jvm, void** p_env, JavaVMInitArgs* vm_args)
```

initializes the Java virtual machine. The function returns

```
0
```

if successful,

```
JNI_ERR
```

on failure. The

`p_jvm`

parameter is filled with a pointer to the invocation API function table, and

`p_env`

is filled with a pointer to the JNI function table.

`jint DestroyJavaVM(JavaVM* jvm)`

destroys the virtual machine. Returns

`0`

on success, a negative number on failure. This function must be called through a virtual machine pointer, that is,

`(*jvm)->DestroyJavaVM(jvm)`

.

12.10 A Complete Example: Accessing the Windows Registry

In this section, I describe a full, working example that covers everything we discussed in this chapter: using native methods with strings, arrays, objects, constructor calls, and error handling. I'll show you how to put a Java platform wrapper around a subset of the ordinary C-based APIs used to work with the Windows registry. Of course, the Windows registry being a Windows-specific feature, such a program is inherently nonportable. For that reason, the standard Java library has no support for the registry, and it makes sense to use native methods to gain access to it.

12.10.1 Overview of the Windows Registry

The Windows registry is a data depository that holds configuration information for the Windows operating system and application programs. It provides a single point for administration and backup of system and application preferences. On the downside, the registry is also a single point of failure—if you mess up the registry, your computer could malfunction or even fail to boot!

We don't suggest that you use the registry to store configuration parameters for your Java programs. The Java preferences API is a better solution (see Volume I, **Chapter 10** for more information). We'll simply use the registry to demonstrate how to wrap a nontrivial native API into a Java class.

The principal tool for inspecting the registry is the *registry editor.* Because of the potential for error by naive but enthusiastic users, there is no icon for launching the registry editor. Instead, start a DOS shell (or open the Start → Run dialog box) and type

```
regedit
```

. **Figure 12.4** shows the registry editor in action.

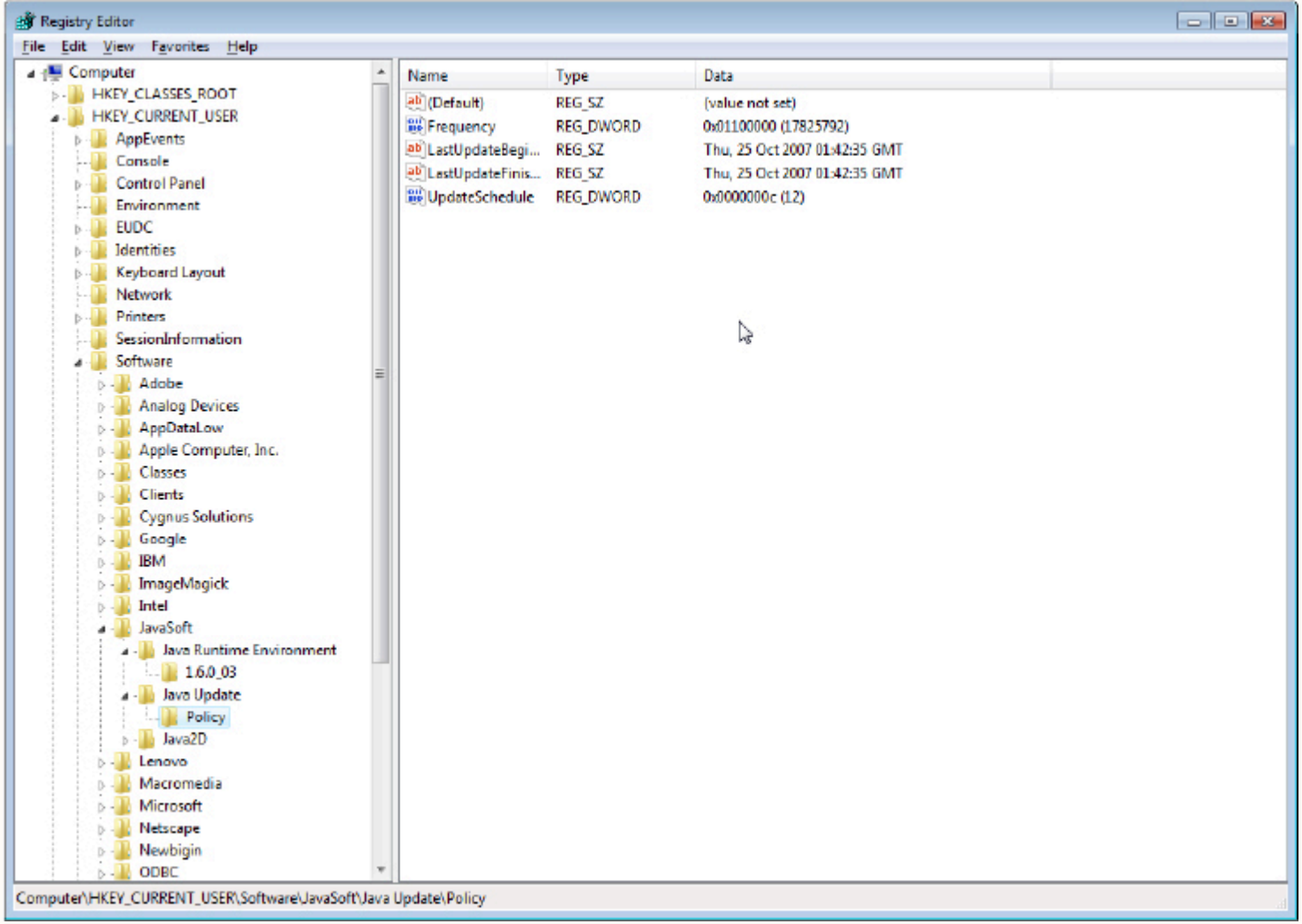


Figure 12.4 The registry editor

The left side shows the keys, which are arranged in a tree structure. Note that each key starts with one of the

```
HKEY
```

nodes like

```
HKEY_CLASSES_ROOT
HKEY_CURRENT_USER
HKEY_LOCAL_MACHINE
. . .
```

The right side shows the name/value pairs associated with a particular key. For example, if you installed Java 17, the key

Click here to view code image

```
HKEY_LOCAL_MACHINE\Software\JavaSoft\Java Runtime Environment
```

contains a name/value pair such as

```
CurrentVersion="17.0_10"
```

In this case, the value is a string. The values can also be integers or arrays of bytes.

12.10.2 A Java Platform Interface for Accessing the Registry

We create a simple interface to access the registry from Java code, and then implement this interface with native code. The interface allows only a few registry operations; to keep the code size down, I omitted some important operations such as adding, deleting, and enumerating keys. (It should be easy to add the remaining registry API functions.)

Even with the limited subset that we supply, you can

Enumerate all names stored in a key
Read the value stored with a name
Set the value stored with a name

Here is the Java class that encapsulates a registry key:

Click here to view code image

```
public class Win32RegKey
{
  public Win32RegKey(int theRoot, String thePath) { . . . }
  public Enumeration names() { . . . }
  public native Object getValue(String name);
  public native void setValue(String name, Object value);

  public static final int HKEY_CLASSES_ROOT = 0×80000000;
  public static final int HKEY_CURRENT_USER = 0×80000001;
  public static final int HKEY_LOCAL_MACHINE = 0×80000002;
  . . .
}
```

The
`names`
method returns an enumeration that holds all the names stored with the key. You can get at them with the familiar
`hasMoreElements/nextElement`
methods. The
`getValue`
method returns an object that is either a string, an
`Integer`
object, or a byte array. The value parameter of the
`setValue`
method must also be of one of these three types.

12.10.3 Implementation of Registry Access Functions as Native Methods

We need to implement three actions:

- Get the value of a name
- Set the value of a name
- Iterate through the names of a key

In this chapter, you have seen essentially all the tools that are required, such as the conversion between Java strings and arrays and those of C. You also saw how to raise a Java exception in case something goes wrong.

Two issues make these native methods more complex than the preceding examples. The
`getValue`
and

`setValue`
methods deal with the type
`Object`
, which can be one of
`String`
,
`Integer`
, or
`byte[]`
. The enumeration object stores the state between successive calls to
`hasMoreElements`
and
`nextElement`
.

Let us first look at the
`getValue`
method. The method (shown in **Listing 12.22**) goes through the following steps:

- Opens the registry key. To read their values, the registry API requires that keys be open.
- Queries the type and size of the value associated with the name.
- Reads the data into a buffer.
- Calls
  `NewStringUTF`
  to create a new string with the value data if the type is
  `REG_SZ`

(a string).
Invokes the
`Integer`
constructor if the type is
`REG_DWORD`
(a 32-bit integer).
Calls
`NewByteArray`
to create a new byte array, then
`SetByteArrayRegion`
to copy the value data into the byte array, if the type is
`REG_BINARY`
.
If the type is none of these or if an error occurred when an API function was called, throws an exception and releases all resources that had been acquired up to that point.
Closes the key and returns the object (
`String`
,
`Integer`
, or
`byte[]`
) that had been created.

As you can see, this example illustrates quite nicely how to generate Java objects of different types.

In this native method, coping with the generic return type is not difficult. The

`jstring`

,

`jobject`

, or

`jarray`

reference is simply returned as a

`jobject`

. However, the

`setValue`

method receives a reference to an

`Object`

and must determine the

`Object`

's exact type to save the

`Object`

as a string, integer, or byte array. We can make this determination by querying the class of the

`value`

object, finding the class references for

`java.lang.String`

,

`java.lang.Integer`

, and

`byte[]`

, and comparing them with the

`IsAssignableFrom`

function.

If

class1

and

class2

are two class references, then the call

Click here to view code image

```
(*env)->IsAssignableFrom(env, class1, class2)
```

returns

JNI_TRUE

when

class1

and

class2

are the same class or when

class1

is a subclass of

class2

. In either case, references to objects of

class1

can be cast to

`class2`

. For example, when

Click here to view code image

```
(*env)->IsAssignableFrom(env, (*env)->GetObjectClass(env, value),
  (*env)->FindClass(env, "[B"))
```

is

`true`

, we know that

`value`

is a byte array.

Here is an overview of the steps in the

`setValue`

method:

1. Open the registry key for writing.
2. Find the type of the value to write.
3. Call `GetStringUTFChars` to get a pointer to the characters if the type is `String`

.

Call the

`intValue`

method to get the integer stored in the wrapper object if the type is

`Integer`

.

Call

`GetByteArrayElements`

to get a pointer to the bytes if the type is

`byte[]`

.

Pass the data and length to the registry.

Close the key.

Release the pointer to the data if the type is

`String`

or

`byte[]`

.

Finally, let us turn to the native methods that enumerate keys. These are methods of the

`Win32RegKeyNameEnumeration`

class (see **Listing 12.21**). When the enumeration process starts, we must open the key. For the duration of the enumeration, we must retain the key handle—that is, the key handle must be stored with the enumeration object. The key handle is of type

`DWORD`

(a 32-bit quantity), so it can be stored in a Java integer. We store it in the

`hkey`

field of the enumeration class. When the enumeration starts, the field is initialized with

`SetIntField`

. Subsequent calls read the value with

`GetIntField`

.

In this example, we store three other data items with the enumeration object. When the enumeration first starts, we can query the registry for the count of name/value pairs and the length of the longest name, which we need so we can allocate C character arrays to hold the names. These values are stored in the

`count`

and

`maxsize`

fields of the enumeration object. Finally, the

`index`

field, initialized with

`-1`

to indicate the start of the enumeration, is set to

`0`

once the other instance fields are initialized, and is incremented after every enumeration step.

Let's walk through the native methods that support the enumeration. The

`hasMoreElements`

method is simple:

Retrieve the

`index`

and `count` fields.
If the index is `-1`, call the `startNameEnumeration` function which opens the key, queries the count and maximum length, and initializes the `hkey`, `count`, `maxsize`, and `index` fields.
Return `JNI_TRUE` if `index` is less than `count`, and `JNI_FALSE` otherwise.

The `nextElement` method needs to work a little harder:

- Retrieve the `index` and `count` fields.
- If the index is `-1`, call the `startNameEnumeration` function, which opens the key, queries the count and maximum length, and initializes the `hkey`, `count`, `maxsize`, and `index` fields.
- If `index` equals `count`

, throw a

`NoSuchElementException`

.

Read the next name from the registry.

Increment

`index`

.

If

`index`

equals

`count`

, close the key.

Before compiling, remember to run

`javac -h`

on both

`Win32RegKey`

and

`Win32RegKeyNameEnumeration`

. The complete command line for the Microsoft compiler is

Click here to view code image

```
cl -I
jdk
\include -I
```

```
jdk
\include\win32 -LD Win32RegKey.c advapi32.lib -FeWin32RegKey.dll
```

With Cygwin, use

Click here to view code image

```
gcc -mno-cygwin -D _int64="long long" -I
jdk
\include -I
jdk
\include\win32 \
  -I c:\cygwin\usr\include\w32api -shared -Wl,--add-stdcall-alias -o Win32RegKey.dll
Win32RegKey.c
```

As the registry API is specific to Windows, this program will not work on other operating systems.

Listing 12.23 shows a program to test our new registry functions. We add three name/value pairs, a string, an integer, and a byte array to the key

Click here to view code image

```
HKEY_CURRENT_USER\Software\JavaSoft\Java Runtime Environment
```

We then enumerate all names of that key and retrieve their values. The program will print

```
Default user=Harry Hacker
Lucky number=13
Small primes=2 3 5 7 11 13
```

Although adding these name/value pairs to that key probably does no harm, you might want to use the registry editor to remove them after running this program.

Listing 12.21

win32reg/Win32RegKey.java

Click here to view code image

```
 1 import java.util.*;
 2
 3 /**
 4  * A Win32RegKey object can be used to get and set values of a registry key in the Windows
 5  * registry.
 6  * @version 1.00 1997-07-01
 7  * @author Cay Horstmann
 8  */
 9 public class Win32RegKey
10 {
11    public static final int HKEY_CLASSES_ROOT = 0x80000000;
12    public static final int HKEY_CURRENT_USER = 0x80000001;
13    public static final int HKEY_LOCAL_MACHINE = 0x80000002;
```

```
14    public static final int HKEY_USERS = 0x80000003;
15    public static final int HKEY_CURRENT_CONFIG = 0x80000005;
16    public static final int HKEY_DYN_DATA = 0x80000006;
17
18    private int root;
19    private String path;
20
21    /**
22     * Gets the value of a registry entry.
23     * @param name the entry name
24     * @return the associated value
25     */
26    public native Object getValue(String name);
27
28    /**
29     * Sets the value of a registry entry.
30     * @param name the entry name
31     * @param value the new value
32     */
33    public native void setValue(String name, Object value);
34
35    /**
36     * Construct a registry key object.
37     * @param theRoot one of HKEY_CLASSES_ROOT, HKEY_CURRENT_USER, HKEY_LOCAL_MACHINE,
38     * HKEY_USERS, HKEY_CURRENT_CONFIG, HKEY_DYN_DATA
39     * @param thePath the registry key path
40     */
41    public Win32RegKey(int theRoot, String thePath)
42    {
43       root = theRoot;
44       path = thePath;
45    }
46
```

```
47    /**
48     * Enumerates all names of registry entries under the path that this object describes.
49     * @return an enumeration listing all entry names
50     */
51    public Enumeration<String> names()
52    {
53       return new Win32RegKeyNameEnumeration(root, path);
54    }
55
56    static
57    {
58       System.loadLibrary("Win32RegKey");
59    }
60 }
61
62 class Win32RegKeyNameEnumeration implements Enumeration<String>
63 {
64    public native String nextElement();
65    public native boolean hasMoreElements();
66    private int root;
67    private String path;
68    private int index = -1;
69    private int hkey = 0;
70    private int maxsize;
71    private int count;
72
73    Win32RegKeyNameEnumeration(int theRoot, String thePath)
74    {
75       root = theRoot;
76       path = thePath;
77    }
78 }
79
```

```
80 class Win32RegKeyException extends RuntimeException
81 {
82    public Win32RegKeyException()
83    {
84    }
85
86    public Win32RegKeyException(String why)
87    {
88       super(why);
89    }
90 }
```

Listing 12.22

win32reg/Win32RegKey.c

Click here to view code image

```
 1 /**
 2    @version 1.00 1997-07-01
 3    @author Cay Horstmann
 4 */
 5
 6 #include "Win32RegKey.h"
 7 #include "Win32RegKeyNameEnumeration.h"
 8 #include <string.h>
 9 #include <stdlib.h>
10 #include <windows.h>
11
12 JNIEXPORT jobject JNICALL Java_Win32RegKey_getValue(
```

```
13       JNIEnv* env, jobject this_obj, jobject name)
14 {
15    const char* cname;
16    jstring path;
17    const char* cpath;
18    HKEY hkey;
19    DWORD type;
20    DWORD size;
21    jclass this_class;
22    jfieldID id_root;
23    jfieldID id_path;
24    HKEY root;
25    jobject ret;
26    char* cret;
27
28    /* get the class */
29    this_class = (*env)->GetObjectClass(env, this_obj);
30
31    /* get the field IDs */
32    id_root = (*env)->GetFieldID(env, this_class, "root", "I");
33    id_path = (*env)->GetFieldID(env, this_class, "path", "Ljava/lang/String;");
34
35    /* get the fields */
36    root = (HKEY) (*env)->GetIntField(env, this_obj, id_root);
37    path = (jstring)(*env)->GetObjectField(env, this_obj, id_path);
38    cpath = (*env)->GetStringUTFChars(env, path, NULL);
39
40    /* open the registry key */
41    if (RegOpenKeyEx(root, cpath, 0, KEY_READ, &hkey) != ERROR_SUCCESS)
42    {
43       (*env)->ThrowNew(env, (*env)->FindClass(env, "Win32RegKeyException"),
44       "Open key failed");
45       (*env)->ReleaseStringUTFChars(env, path, cpath);
```

```
46       return NULL;
47    }
48
49    (*env)->ReleaseStringUTFChars(env, path, cpath);
50    cname = (*env)->GetStringUTFChars(env, name, NULL);
51
52    /* find the type and size of the value */
53    if (RegQueryValueEx(hkey, cname, NULL, &type, NULL, &size) != ERROR_SUCCESS)
54    {
55       (*env)->ThrowNew(env, (*env)->FindClass(env, "Win32RegKeyException"),
56             "Query value key failed");
57       RegCloseKey(hkey);
58       (*env)->ReleaseStringUTFChars(env, name, cname);
59       return NULL;
60    }
61
62    /* get memory to hold the value */
63    cret = (char*)malloc(size);
64
65    /* read the value */
66    if (RegQueryValueEx(hkey, cname, NULL, &type, cret, &size) != ERROR_SUCCESS)
67    {
68       (*env)->ThrowNew(env, (*env)->FindClass(env, "Win32RegKeyException"),
69             "Query value key failed");
70       free(cret);
71       RegCloseKey(hkey);
72       (*env)->ReleaseStringUTFChars(env, name, cname);
73       return NULL;
74    }
75
76    /* depending on the type, store the value in a string,
77    integer, or byte array */
78    if (type == REG_SZ)
```

```
 79    {
 80       ret = (*env)->NewStringUTF(env, cret);
 81    }
 82    else if (type == REG_DWORD)
 83    {
 84       jclass class_Integer = (*env)->FindClass(env, "java/lang/Integer");
 85       /* get the method ID of the constructor */
 86       jmethodID id_Integer = (*env)->GetMethodID(env, class_Integer, "<init>", "(I)V");
 87       int value = *(int*) cret;
 88       /* invoke the constructor */
 89       ret = (*env)->NewObject(env, class_Integer, id_Integer, value);
 90    }
 91    else if (type == REG_BINARY)
 92    {
 93       ret = (*env)->NewByteArray(env, size);
 94       (*env)->SetByteArrayRegion(env, (jarray) ret, 0, size, cret);
 95    }
 96    else
 97    {
 98       (*env)->ThrowNew(env, (*env)->FindClass(env, "Win32RegKeyException"),
 99             "Unsupported value type");
100       ret = NULL;
101    }
102
103    free(cret);
104    RegCloseKey(hkey);
105    (*env)->ReleaseStringUTFChars(env, name, cname);
106
107    return ret;
108 }
109
110 JNIEXPORT void JNICALL Java_Win32RegKey_setValue(JNIEnv* env, jobject this_obj,
111       jstring name, jobject value)
```

```
112 {
113    const char* cname;
114    jstring path;
115    const char* cpath;
116    HKEY hkey;
117    DWORD type;
118    DWORD size;
119    jclass this_class;
120    jclass class_value;
121    jclass class_Integer;
122    jfieldID id_root;
123    jfieldID id_path;
124    HKEY root;
125    const char* cvalue;
126    int ivalue;
127
128    /* get the class */
129    this_class = (*env)->GetObjectClass(env, this_obj);
130
131    /* get the field IDs */
132    id_root = (*env)->GetFieldID(env, this_class, "root", "I");
133    id_path = (*env)->GetFieldID(env, this_class, "path", "Ljava/lang/String;");
134
135    /* get the fields */
136    root = (HKEY)(*env)->GetIntField(env, this_obj, id_root);
137    path = (jstring)(*env)->GetObjectField(env, this_obj, id_path);
138    cpath = (*env)->GetStringUTFChars(env, path, NULL);
139
140    /* open the registry key */
141    if (RegOpenKeyEx(root, cpath, 0, KEY_WRITE, &hkey) != ERROR_SUCCESS)
142    {
143       (*env)->ThrowNew(env, (*env)->FindClass(env, "Win32RegKeyException"),
144             "Open key failed");
```

```
145       (*env)->ReleaseStringUTFChars(env, path, cpath);
146       return;
147    }
148
149    (*env)->ReleaseStringUTFChars(env, path, cpath);
150    cname = (*env)->GetStringUTFChars(env, name, NULL);
151
152    class_value = (*env)->GetObjectClass(env, value);
153    class_Integer = (*env)->FindClass(env, "java/lang/Integer");
154    /* determine the type of the value object */
155    if ((*env)->IsAssignableFrom(env, class_value, (*env)->FindClass(env, "java/lang/Strin
g")))
156    {
157       /* it is a string--get a pointer to the characters */
158       cvalue = (*env)->GetStringUTFChars(env, (jstring) value, NULL);
159       type = REG_SZ;
160       size = (*env)->GetStringLength(env, (jstring) value) + 1;
161    }
162    else if ((*env)->IsAssignableFrom(env, class_value, class_Integer))
163    {
164       /* it is an integer--call intValue to get the value */
165       jmethodID id_intValue = (*env)->GetMethodID(env, class_Integer, "intValue", "()I");
166       ivalue = (*env)->CallIntMethod(env, value, id_intValue);
167       type = REG_DWORD;
168       cvalue = (char*)&ivalue;
169       size = 4;
170    }
171    else if ((*env)->IsAssignableFrom(env, class_value, (*env)->FindClass(env, "[B")))
172    {
173       /* it is a byte array--get a pointer to the bytes */
174       type = REG_BINARY;
175       cvalue = (char*)(*env)->GetByteArrayElements(env, (jarray) value, NULL);
176       size = (*env)->GetArrayLength(env, (jarray) value);
```

```
177    }
178    else
179    {
180       /* we don't know how to handle this type */
181       (*env)->ThrowNew(env, (*env)->FindClass(env, "Win32RegKeyException"),
182             "Unsupported value type");
183       RegCloseKey(hkey);
184       (*env)->ReleaseStringUTFChars(env, name, cname);
185       return;
186    }
187
188    /* set the value */
189    if (RegSetValueEx(hkey, cname, 0, type, cvalue, size) != ERROR_SUCCESS)
190    {
191       (*env)->ThrowNew(env, (*env)->FindClass(env, "Win32RegKeyException"),
192             "Set value failed");
193    }
194
195    RegCloseKey(hkey);
196    (*env)->ReleaseStringUTFChars(env, name, cname);
197
198    /* if the value was a string or byte array, release the pointer */
199    if (type == REG_SZ)
200    {
201       (*env)->ReleaseStringUTFChars(env, (jstring) value, cvalue);
202    }
203    else if (type == REG_BINARY)
204    {
205       (*env)->ReleaseByteArrayElements(env, (jarray) value, (jbyte*) cvalue, 0);
206    }
207 }
208
209 /* helper function to start enumeration of names */
```

```
210 static int startNameEnumeration(JNIEnv* env, jobject this_obj, jclass this_class)
211 {
212    jfieldID id_index;
213    jfieldID id_count;
214    jfieldID id_root;
215    jfieldID id_path;
216    jfieldID id_hkey;
217    jfieldID id_maxsize;
218
219    HKEY root;
220    jstring path;
221    const char* cpath;
222    HKEY hkey;
223    DWORD maxsize = 0;
224    DWORD count = 0;
225
226    /* get the field IDs */
227    id_root = (*env)->GetFieldID(env, this_class, "root", "I");
228    id_path = (*env)->GetFieldID(env, this_class, "path", "Ljava/lang/String;");
229    id_hkey = (*env)->GetFieldID(env, this_class, "hkey", "I");
230    id_maxsize = (*env)->GetFieldID(env, this_class, "maxsize", "I");
231    id_index = (*env)->GetFieldID(env, this_class, "index", "I");
232    id_count = (*env)->GetFieldID(env, this_class, "count", "I");
233
234    /* get the field values */
235    root = (HKEY)(*env)->GetIntField(env, this_obj, id_root);
236    path = (jstring)(*env)->GetObjectField(env, this_obj, id_path);
237    cpath = (*env)->GetStringUTFChars(env, path, NULL);
238
239    /* open the registry key */
240    if (RegOpenKeyEx(root, cpath, 0, KEY_READ, &hkey) != ERROR_SUCCESS)
241    {
242       (*env)->ThrowNew(env, (*env)->FindClass(env, "Win32RegKeyException"),
```

```
243            "Open key failed");
244       (*env)->ReleaseStringUTFChars(env, path, cpath);
245       return -1;
246    }
247    (*env)->ReleaseStringUTFChars(env, path, cpath);
248
249    /* query count and max length of names */
250    if (RegQueryInfoKey(hkey, NULL, NULL, NULL, NULL, NULL, NULL, &count, &maxsize,
251          NULL, NULL, NULL) != ERROR_SUCCESS)
252    {
253       (*env)->ThrowNew(env, (*env)->FindClass(env, "Win32RegKeyException"),
254             "Query info key failed");
255       RegCloseKey(hkey);
256       return -1;
257    }
258
259    /* set the field values */
260    (*env)->SetIntField(env, this_obj, id_hkey, (DWORD) hkey);
261    (*env)->SetIntField(env, this_obj, id_maxsize, maxsize + 1);
262    (*env)->SetIntField(env, this_obj, id_index, 0);
263    (*env)->SetIntField(env, this_obj, id_count, count);
264    return count;
265 }
266
267 JNIEXPORT jboolean JNICALL Java_Win32RegKeyNameEnumeration_hasMoreElements(JNIEnv* env,
268       jobject this_obj)
269 {
270    jclass this_class;
271    jfieldID id_index;
272    jfieldID id_count;
273    int index;
274    int count;
275    /* get the class */
```

```
276    this_class = (*env)->GetObjectClass(env, this_obj);
277
278    /* get the field IDs */
279    id_index = (*env)->GetFieldID(env, this_class, "index", "I");
280    id_count = (*env)->GetFieldID(env, this_class, "count", "I");
281
282    index = (*env)->GetIntField(env, this_obj, id_index);
283    if (index == -1) /* first time */
284    {
285       count = startNameEnumeration(env, this_obj, this_class);
286       index = 0;
287    }
288    else
289       count = (*env)->GetIntField(env, this_obj, id_count);
290    return index < count;
291}
292
293 JNIEXPORT jobject JNICALL Java_Win32RegKeyNameEnumeration_nextElement(JNIEnv* env,
294       jobject this_obj)
295 {
296    jclass this_class;
297    jfieldID id_index;
298    jfieldID id_hkey;
299    jfieldID id_count;
300    jfieldID id_maxsize;
301
302    HKEY hkey;
303    int index;
304    int count;
305    DWORD maxsize;
306
307    char* cret;
308    jstring ret;
```

```
309
310    /* get the class */
311    this_class = (*env)->GetObjectClass(env, this_obj);
312
313    /* get the field IDs */
314    id_index = (*env)->GetFieldID(env, this_class, "index", "I");
315    id_count = (*env)->GetFieldID(env, this_class, "count", "I");
316    id_hkey = (*env)->GetFieldID(env, this_class, "hkey", "I");
317    id_maxsize = (*env)->GetFieldID(env, this_class, "maxsize", "I");
318
319    index = (*env)->GetIntField(env, this_obj, id_index);
320    if (index == -1) /* first time */
321    {
322       count = startNameEnumeration(env, this_obj, this_class);
323       index = 0;
324    }
325    else
326       count = (*env)->GetIntField(env, this_obj, id_count);
327
328    if (index >= count) /* already at end */
329    {
330       (*env)->ThrowNew(env, (*env)->FindClass(env, "java/util/NoSuchElementException"),
331             "past end of enumeration");
332       return NULL;
333    }
334
335    maxsize = (*env)->GetIntField(env, this_obj, id_maxsize);
336    hkey = (HKEY)(*env)->GetIntField(env, this_obj, id_hkey);
337    cret = (char*)malloc(maxsize);
338
339    /* find the next name */
340    if (RegEnumValue(hkey, index, cret, &maxsize, NULL, NULL, NULL, NULL) != ERROR_SUCCES
S)
```

```
341    {
342       (*env)->ThrowNew(env, (*env)->FindClass(env, "Win32RegKeyException"),
343             "Enum value failed");
344       free(cret);
345       RegCloseKey(hkey);
346       (*env)->SetIntField(env, this_obj, id_index, count);
347       return NULL;
348    }
349
350    ret = (*env)->NewStringUTF(env, cret);
351    free(cret);
352
353    /* increment index */
354    index++;
355    (*env)->SetIntField(env, this_obj, id_index, index);
356
357    if (index == count) /* at end */
358    {
359       RegCloseKey(hkey);
360    }
361
362    return ret;
363 }
```

Listing 12.23

win32reg/Win32RegKeyTest.java

Click here to view code image

```
 1 import java.util.*;
 2
 3 /**
 4    @version 1.04 2021-05-30
 5    @author Cay Horstmann
 6 */
 7 public class Win32RegKeyTest
 8 {
 9    public static void main(String[] args)
10    {
11       var key = new Win32RegKey(
12          Win32RegKey.HKEY_CURRENT_USER, "Software\\JavaSoft\\Java Runtime Environment");
13
14       key.setValue("Default user", "Harry Hacker");
15       key.setValue("Lucky number", Integer.valueOf(13));
16       key.setValue("Small primes", new byte[] { 2, 3, 5, 7, 11 });
17
18       Enumeration<String> e = key.names();
19
20       while (e.hasMoreElements())
21       {
22          String name = e.nextElement();
23          System.out.print(name + "=");
24
25          Object value = key.getValue(name);
26
27          if (value instanceof byte[] bytes)
28             for (byte b : bytes) System.out.print((b & 0xFF) + " ");
29          else
30             System.out.print(value);
31
```

```
32          System.out.println();
33       }
34    }
35 }
```

Type Inquiry Functions

`jboolean IsAssignableFrom(JNIEnv *env, jclass cl1, jclass cl2)`

returns

`JNI_TRUE`

if objects of the first class can be assigned to objects of the second class, and

`JNI_FALSE`

otherwise. This tests if the classes are the same, or

`cl1`

is a subclass of

`cl2`

, or

`cl2`

represents an interface implemented by

`cl1`

or one of its superclasses.

`jclass GetSuperclass(JNIEnv *env, jclass cl)`

returns the superclass of a class. If

`cl`

represents the class

`Object`

or an interface, returns

`NULL`

.

12.11 Foreign Functions: A Glimpse into the Future

When using JNI, you have to write C code to access Java data structures, call the desired C functions, and to convert the result back to Java. Then you have to link the C code into platform-specific libraries. An API for accessing "foreign" functions and memory, developed under the code name "Project Panama," allows you to write this code in Java.

In Java 17, the API is a preview feature, and the details will change before the final release. The program in **Listing 12.24** is a very simple demonstration of the preview API.

The program simply calls the

`printf`

function. No additional C code is necessary. The function is invoked through a Java

`MethodHandle`

, which is normally used to call Java functions.

To access C functions, you first create a

`CLinker`

object. The

`CLinker`

can produce a Java method handle for a C function, given its address and type description. The details of this API are in flux, so let us not dwell on them.

The API has methods for translating Java objects into memory blocks that you can pass to the C function. In the demo, you need to provide a

`char*`

with the characters from a Java

`String`

. The

`CLinker`

provides this translation.

The memory is allocated in a

`ResourceScope`

, so that its deallocation can be managed. Here, we use a *confined* scope. It is autocloseable, and the

`close`

method deallocates all allocated memory.

As you can see, this API is much simpler and more convenient than JNI. This demo only scratches the surface of the API. There are many methods for reading, writing, and managing memory blocks. You can also pass Java functions as callbacks to C functions.

To compile the demo program, use these options:

Click here to view code image

```
javac --enable-preview --source 17 --add-modules \
```

```
jdk.incubator.foreign panama/PanamaDemo.java
```

The foreign function and memory API is inherently unsafe because it provides access to unmanaged memory. You use a runtime flag to permit a module to use the API. Typically, one would place the bridge code between a Java API and a C library into a separate module. However, in the demo, the native access happens in the unnamed module. Here is the command line to run it:

Click here to view code image

```
java --add-modules jdk.incubator.foreign \
  --enable-native-access=ALL-UNNAMED panama.PanamaDemo
```

For now, you will still need to use JNI as a stable API for interoperating with native code. As the foreign function and memory API matures, it will provide a superior replacement for JNI.

Listing 12.24

panama/PanamaDemo.java

Click here to view code image

```
1 package panama;
2
3 import java.lang.reflect.*;
4 import java.lang.invoke.*;
5 import jdk.incubator.foreign.*;
```

```
 6 import static jdk.incubator.foreign.MemoryLayouts.*;
 7
 8 /*
 9
10 javac --enable-preview --source 17 --add-modules jdk.incubator.foreign panama/PanamaDemo.j
ava
11
12 java --add-modules jdk.incubator.foreign --enable-native-access=ALL-UNNAMED panama.PanamaD
emo
13
14 */
15
16 public class PanamaDemo
17 {
18    public static void main(String[] args) throws Throwable
19    {
20       CLinker linker = CLinker.getInstance();
21       MethodHandle printf = linker.downcallHandle(
22          CLinker.systemLookup().lookup("printf").get(),
23          MethodType.methodType(int.class, MemoryAddress.class),
24          FunctionDescriptor.of(CLinker.C_INT, CLinker.C_POINTER));
25
26       try (ResourceScope scope = ResourceScope.newConfinedScope())
27       {
28          var cString = CLinker.toCString("Hello, World!\n", scope);
29          int result = (int) printf.invoke(cString.address());
30          System.out.println("Printed %d characters.".formatted(result));
31       }
32    }
33 }
```

You have now reached the end of the second volume of *Core Java*, completing a long journey in which you encountered many advanced APIs. I started out with topics that every Java programmer needs to know: streams, XML, networking, databases, and internationalization. I concluded with very technical chapters on security, annotation processing, advanced graphics, and native methods. I hope that you enjoyed your tour through the vast breadth of the Java APIs, and that you will be able to apply your newly gained knowledge in your projects.

Previous chapter
< Chapter 11. Advanced Swing and Graphics

Next chapter
Index >